

COS30082

Applied Machine Learning



Week 8

Generative Deep Learning

What is Generative models?

- In supervised learning, we have data x and target output (label) y , and the goal is to learn a function to map x to y e.g. regression, classification, object detection.
- While in unsupervised learning, there are no labels and the goal is to find some underlying hidden structure of the data e.g. clustering, dimensionality reduction, feature learning.
- A Generative Model is a powerful way of **learning any kind of data distribution using unsupervised learning**.
- All types of generative models aim at learning the true data distribution of the training set so as to **generate new data points** with some variations.

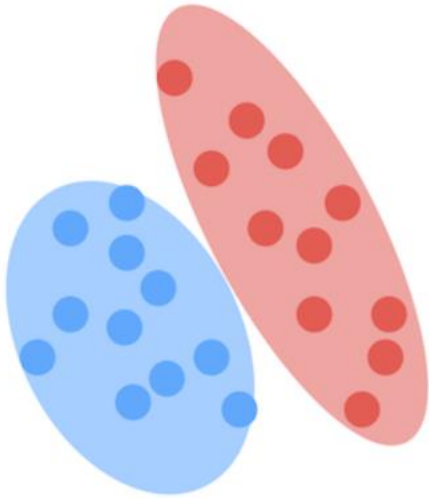
Generative and Discriminative model

A story of lion and elephant

- A father has two kids, **Kid A** and **Kid B**. Kid A has a special character whereas he can learn everything in depth. Kid B have a special character whereas he can only learn the differences between what he saw.
- One fine day, The father takes two of his kids (Kid A and Kid B) to a zoo. This zoo is a very small one and has only two kinds of animals say a lion and an elephant. After they came out of the zoo, the father showed them an animal and asked both of them “is this animal a lion or an elephant?”
- The Kid A, the kid suddenly draw the image of lion and elephant in a piece of paper based on what he saw inside the zoo. He compared both the images with the animal standing before and answered based on the closest match of image & animal, he answered: “The animal is Lion”.
- The Kid B knows only the differences, based on different properties learned, he answered: “The animal is a Lion”.
- Here, we can see both of them is finding the kind of animal, but the way of learning and the way of finding answer is entirely different. In Machine Learning, We generally call Kid A as a Generative Model & Kid B as a Discriminative Model.

Generative and Discriminative model

Generative model



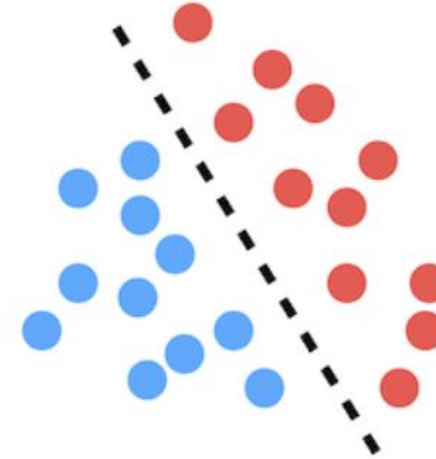
Informally:

Generative models can generate new data instances.

Formally:

Generative models capture the joint probability $p(X, Y)$, or just $p(X)$ if there are no labels.

Discriminative model



Informally:

Discriminative models discriminate between different kinds of data instances.

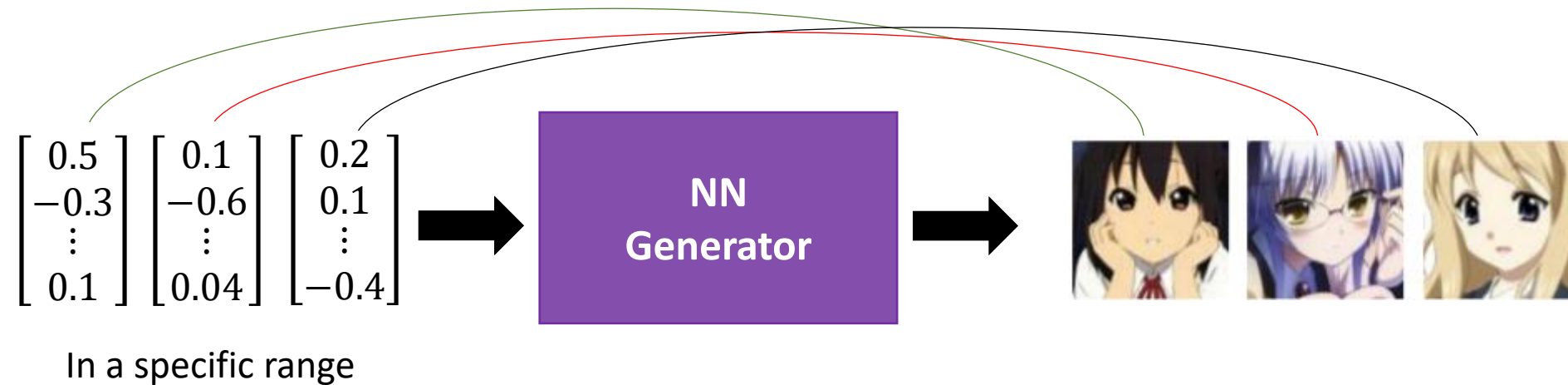
Formally:

Discriminative models capture the conditional probability $p(Y | X)$.

Introduction of Generative Adversarial network (GAN)

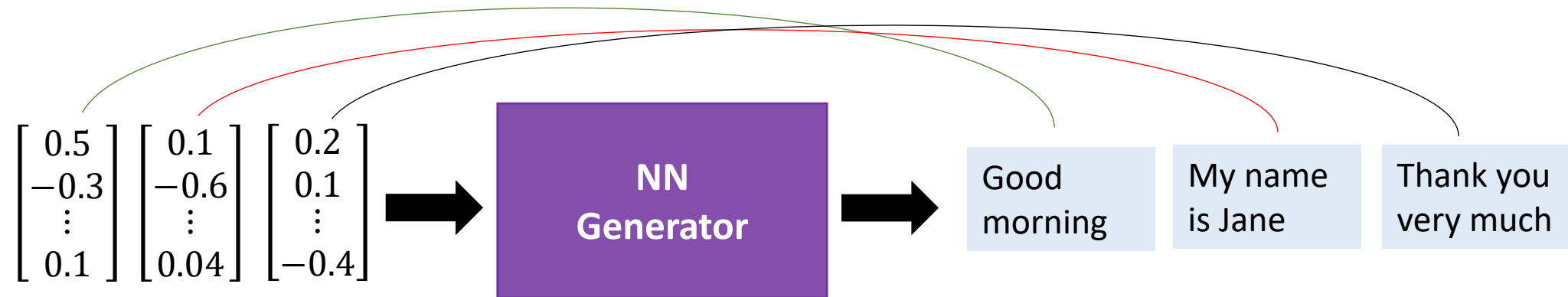
Basic Idea of GAN - Generator

- Image generation

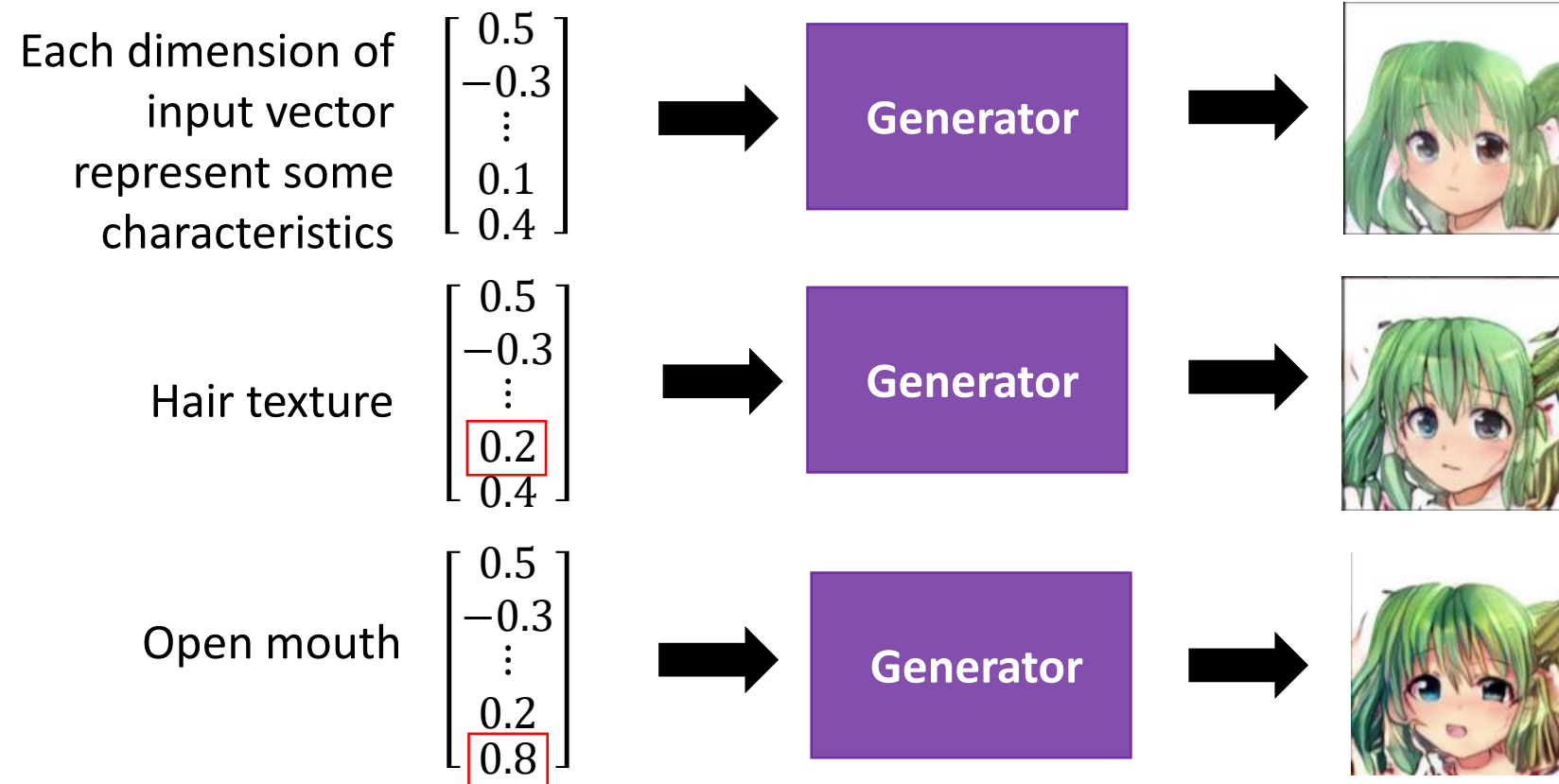
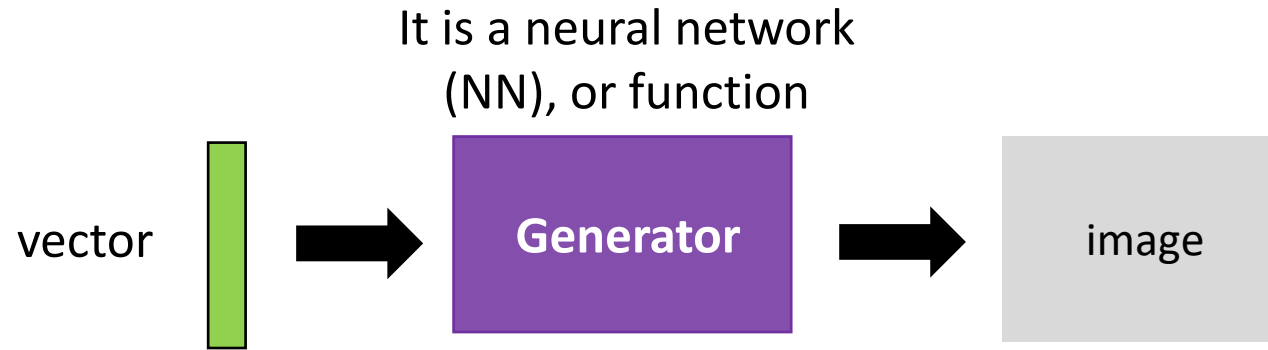


Basic Idea of GAN - Generator

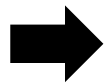
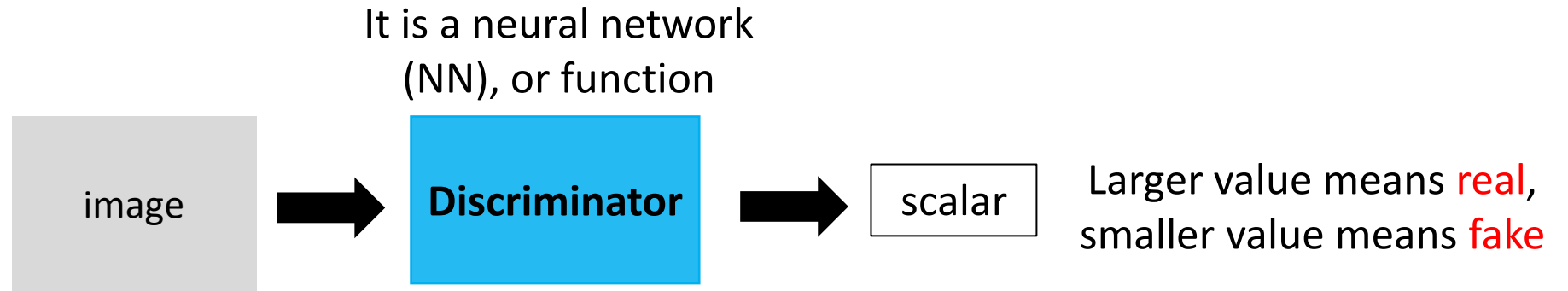
- Sentence generation



Basic Idea of GAN - Generator



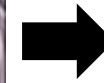
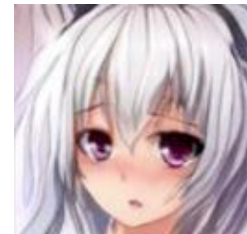
Basic Idea of GAN - Discriminator



Discriminator



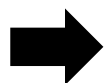
1.0



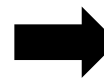
Discriminator



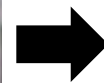
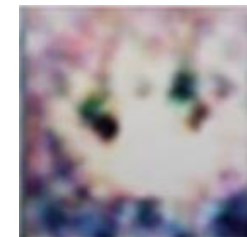
1.0



Discriminator



0.1



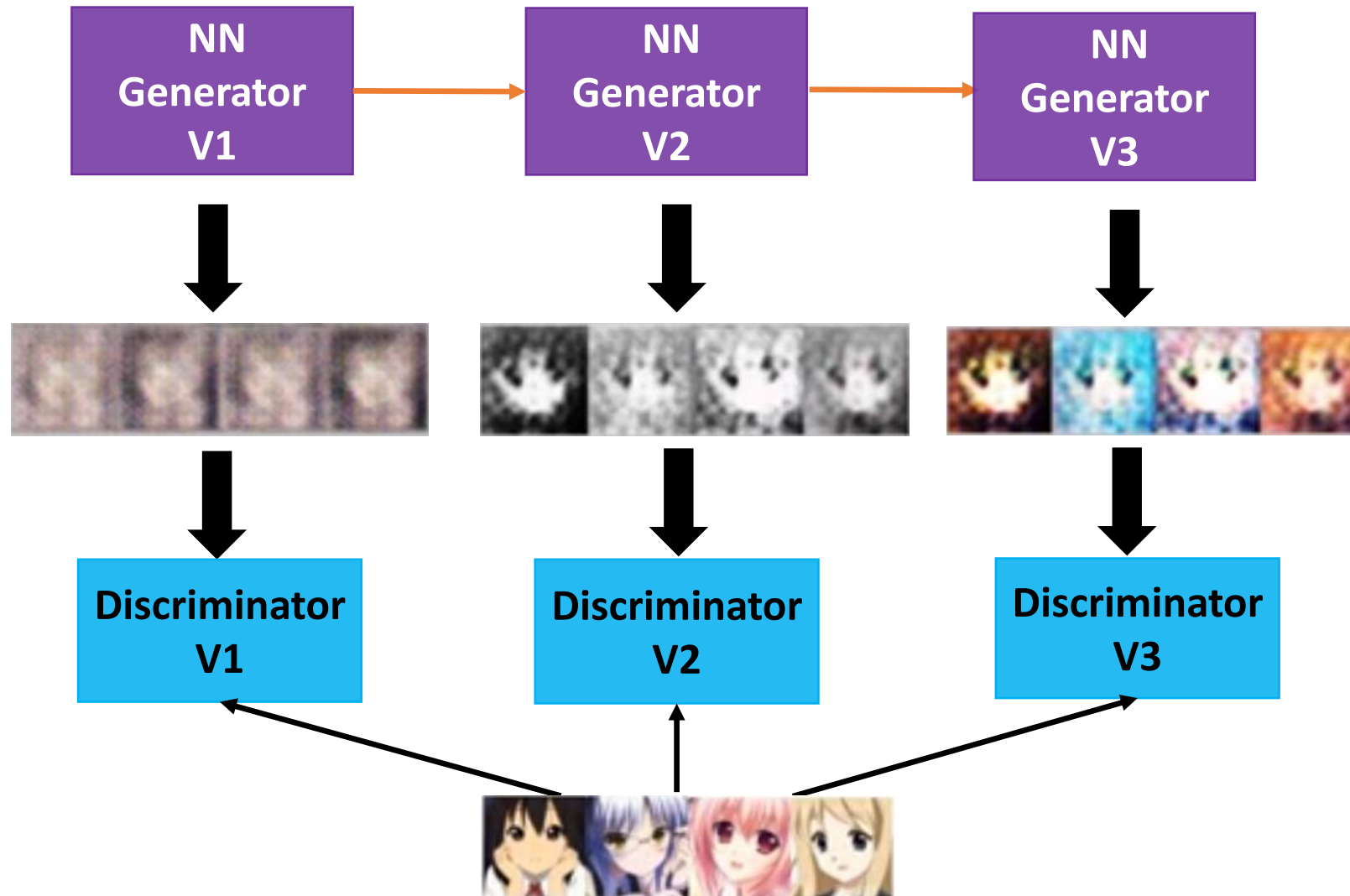
Discriminator



0.1

Generator + Discriminator

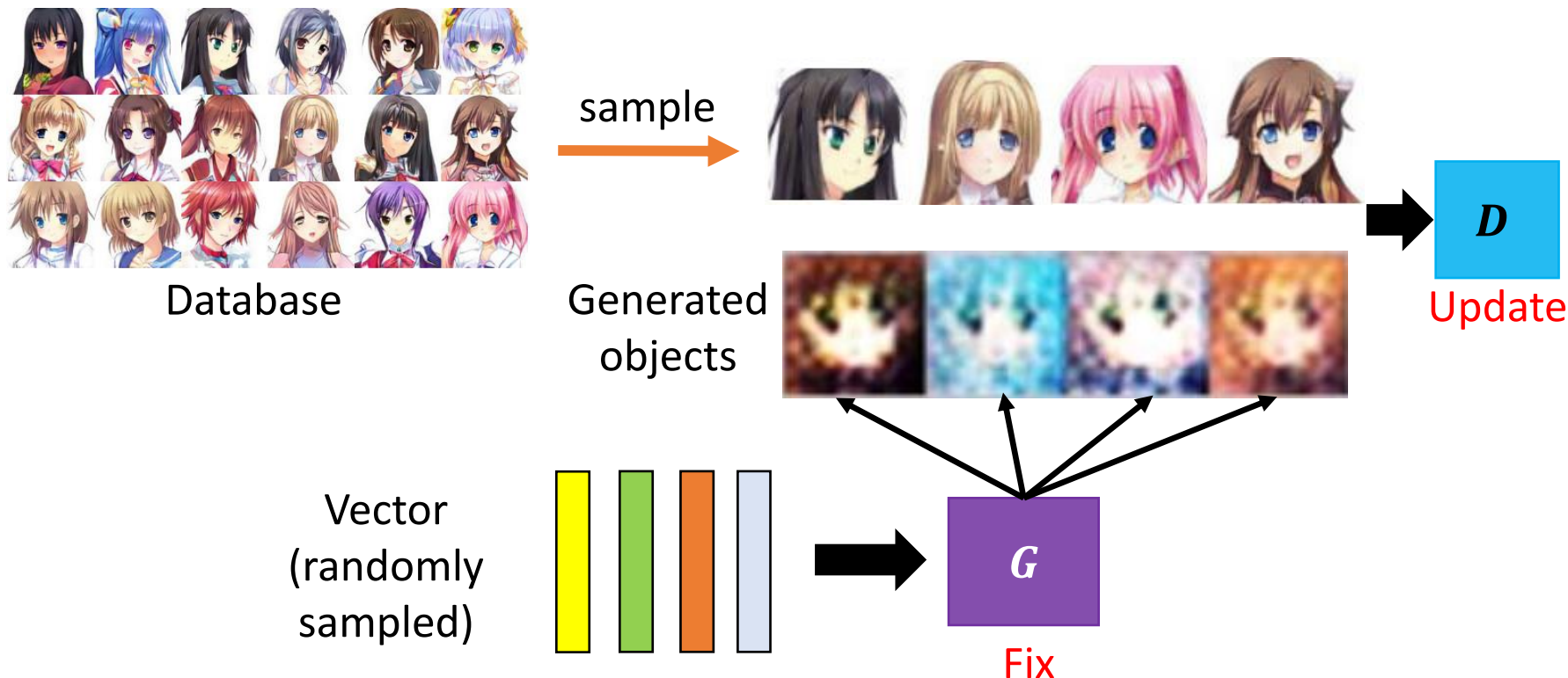
Example: Manga generation using GAN



Training algorithm overview

- Initialize generator and discriminator
- In each training iteration:

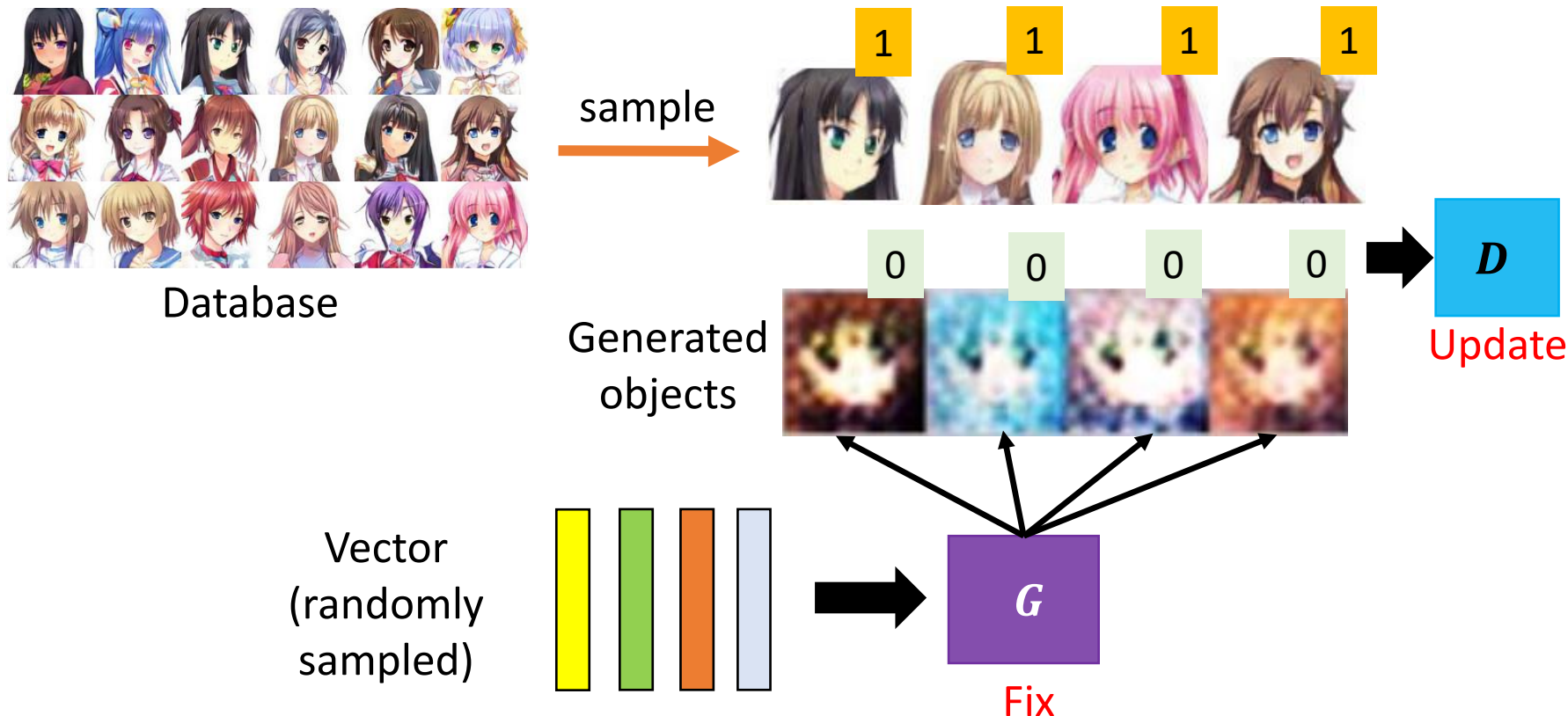
Step 1: Fix generator G and update discriminator D .



Training algorithm overview

- Initialize generator and discriminator
- In each training iteration:

Step 1: Fix generator G and update discriminator D



Discriminator learns to assign high scores to real objects and low scores to generated objects

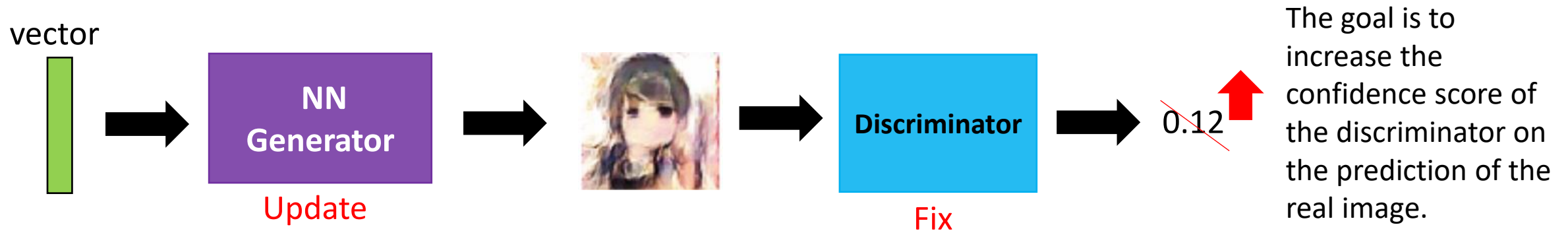
Training algorithm overview

- Initialize generator and discriminator
- In each training iteration:

Generator

Discriminator

Step 2: Fix discriminator D and update generator G
Generator learns to “fool” the discriminator



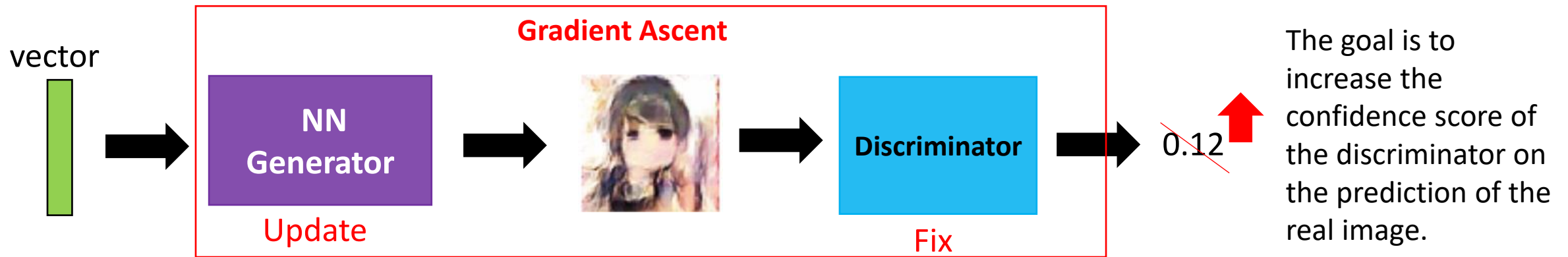
Training algorithm overview

- Initialize generator and discriminator
- In each training iteration:

Generator

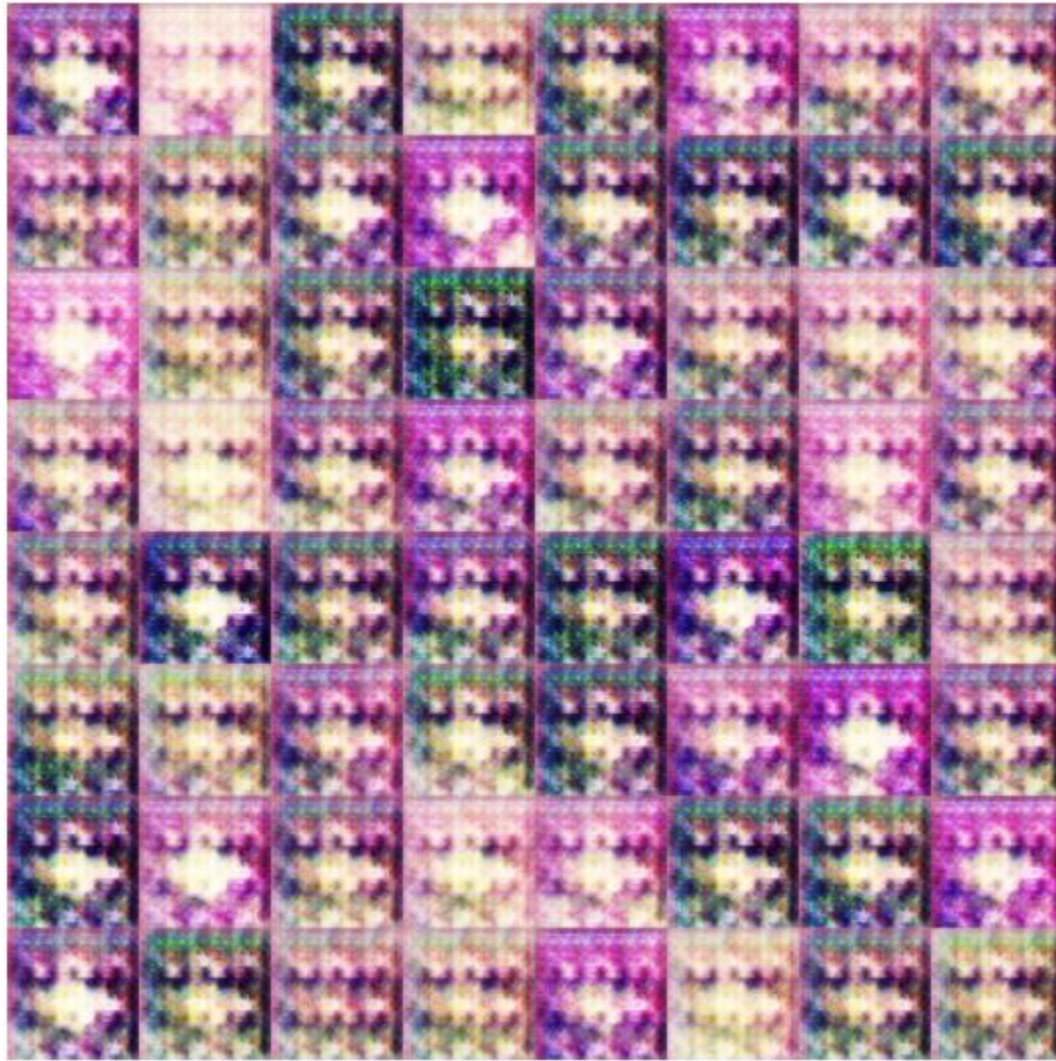
Discriminator

Step 2: Fix discriminator D and update generator G
Generator learns to “fool” the discriminator



Both modules are actually connected forming a large network

Manga generation results



1st epoch



10th epoch

Manga generation results



200th epoch



300th epoch

GAN as Structured Learning

- Machine learning is to find a function f
$$f: x \rightarrow y$$
- Regression: output a scalar
- Classification: output a “class” (one-hot vector)
- **Structured Learning/Prediction:** output a sequence, a matrix, a graph, a tree..
 - Output is composed of component with dependencies

Examples of structured learning

Output Sequence: $f: x \rightarrow y$

- Machine Translation

$x =$ "She very loves cooking" $y =$ "彼女は料理が大好きです"
(English sentence) (Japanese sentence)

- Speech recognition

$x =$  $y =$ "The generator learns to generate plausible data"

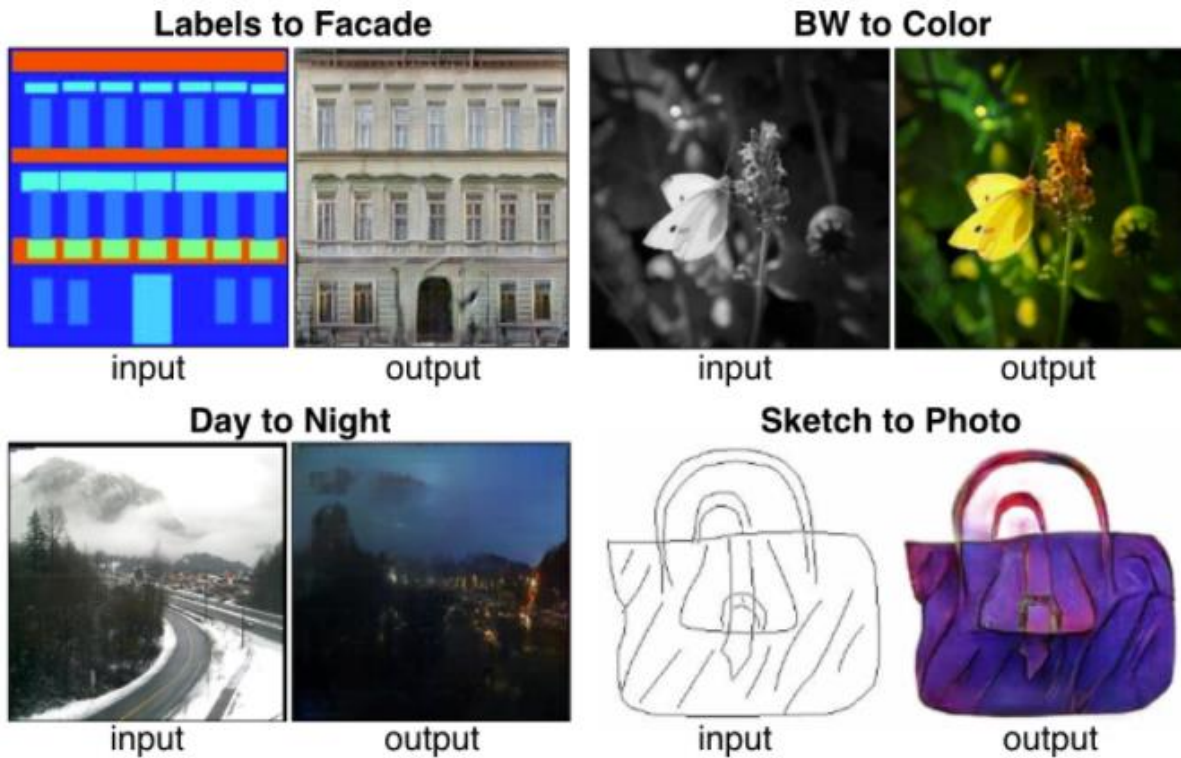
- Chat-box

$x =$ "Hello, how are you?" $y =$ "Thank you, I am fine."
(user says) (machine response)

Examples of structured learning

Output Matrix: $f: x \rightarrow y$

- Image to Image translation



- Text to Image translation

This small bird has a yellow crown and a white belly.

This bird has a blue crown with white throat and brown secondaries.

This bird has a red head, throat and chest, with a white belly.



Why structured learning challenging?

- **One shot/Zero-shot Learning:**
 - In classification, each class has some examples.
 - In structure learning,
 - If you consider each possible output as a “class”...
 - Since the output space is huge, most “classes” do not have any training data
 - Machine has to create new stuff during testing
 - Need more intelligence

Why structured learning challenging?

- **Machine has to learn to do planning**
 - Machine generates objects component-by-component, but it should have a big picture in its mind.
 - Because the output components have dependency, they should be considered globally.
- For example, a pixel of image might depend on its neighboring pixels, the next word in a sentence is dependent on the sequence of the previous words.

GAN as structured learning solution

Generator

Learn to generate
the object at the
component level

**Bottom
Up**



Discriminator

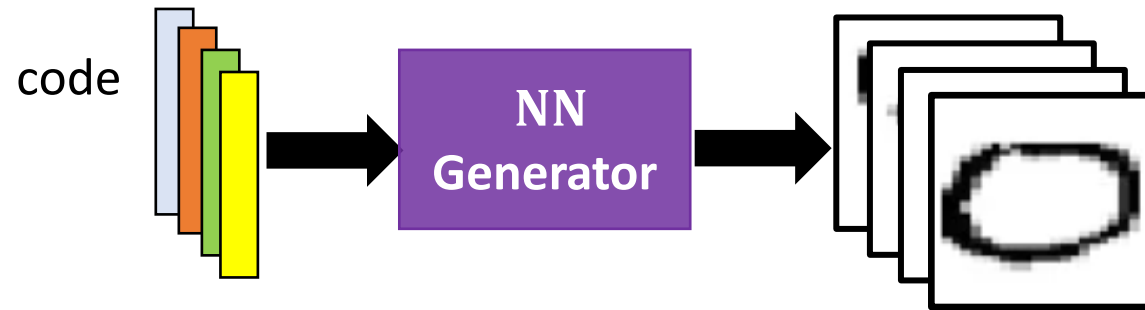
Evaluating the
whole object, and
find the best one

**Top
down**

**Generative
Adversarial
Network (GAN)**

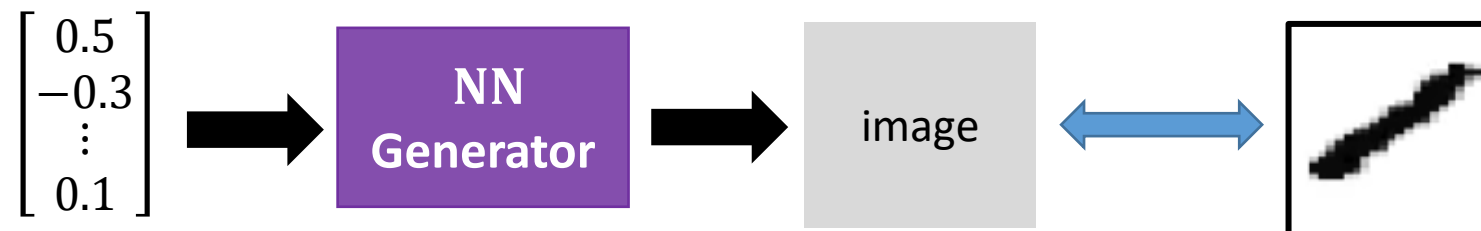
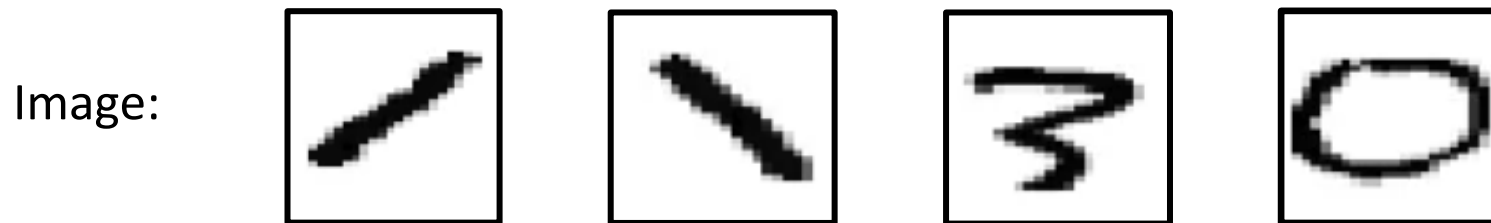
**Can Generator learn itself without
depending on Discriminator?**

Generator



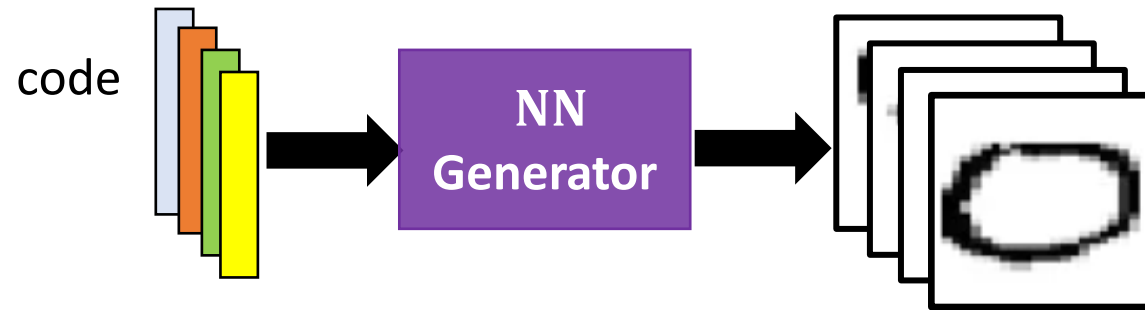
How to generate these codes?

Code: $\begin{bmatrix} 0.5 \\ -0.3 \\ \vdots \\ 0.1 \end{bmatrix}$ $\begin{bmatrix} 0.5 \\ -0.3 \\ \vdots \\ 0.2 \end{bmatrix}$ $\begin{bmatrix} 0.1 \\ -0.5 \\ \vdots \\ 0.5 \end{bmatrix}$ $\begin{bmatrix} 0.3 \\ 0.7 \\ \vdots \\ -0.2 \end{bmatrix}$



As close as possible

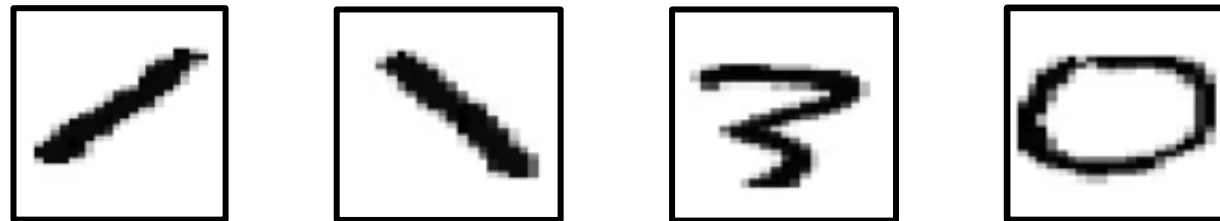
Generator



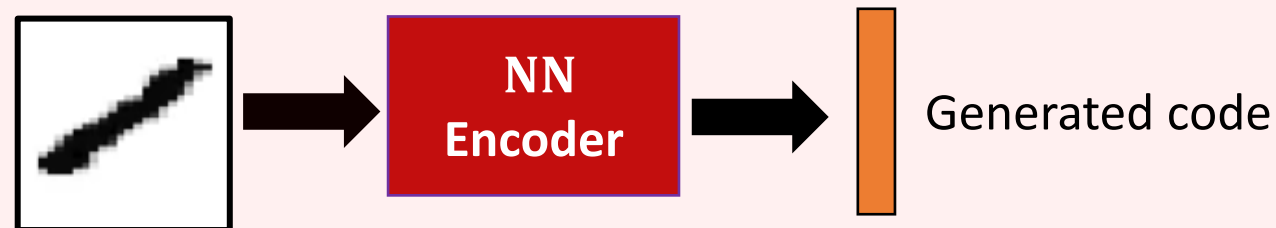
How to generate these codes?

Code: $\begin{bmatrix} 0.5 \\ -0.3 \\ \vdots \\ 0.1 \end{bmatrix}$ $\begin{bmatrix} 0.5 \\ -0.3 \\ \vdots \\ 0.2 \end{bmatrix}$ $\begin{bmatrix} 0.1 \\ -0.5 \\ \vdots \\ 0.5 \end{bmatrix}$ $\begin{bmatrix} 0.3 \\ 0.7 \\ \vdots \\ -0.2 \end{bmatrix}$

Image:

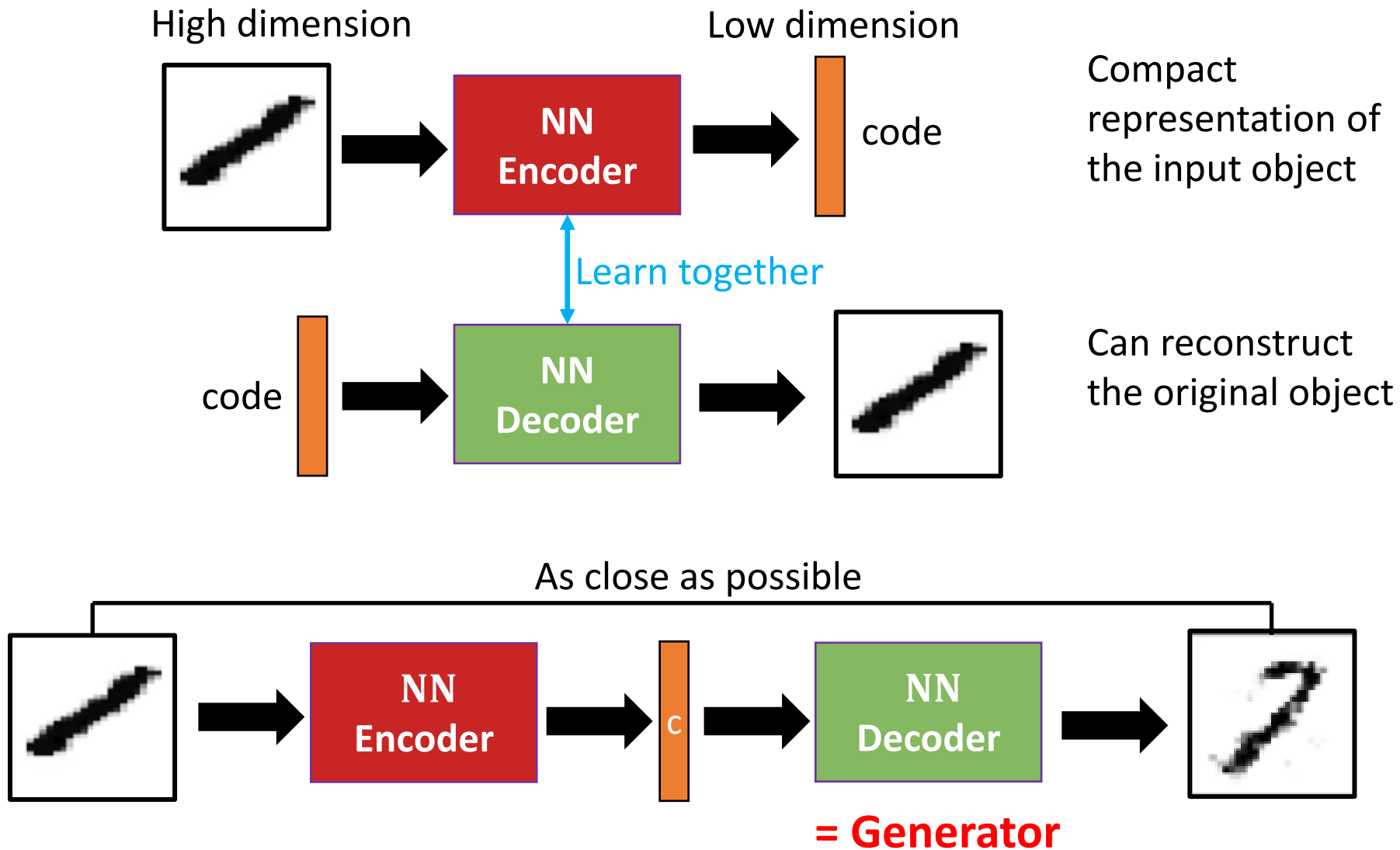


You can use encoder

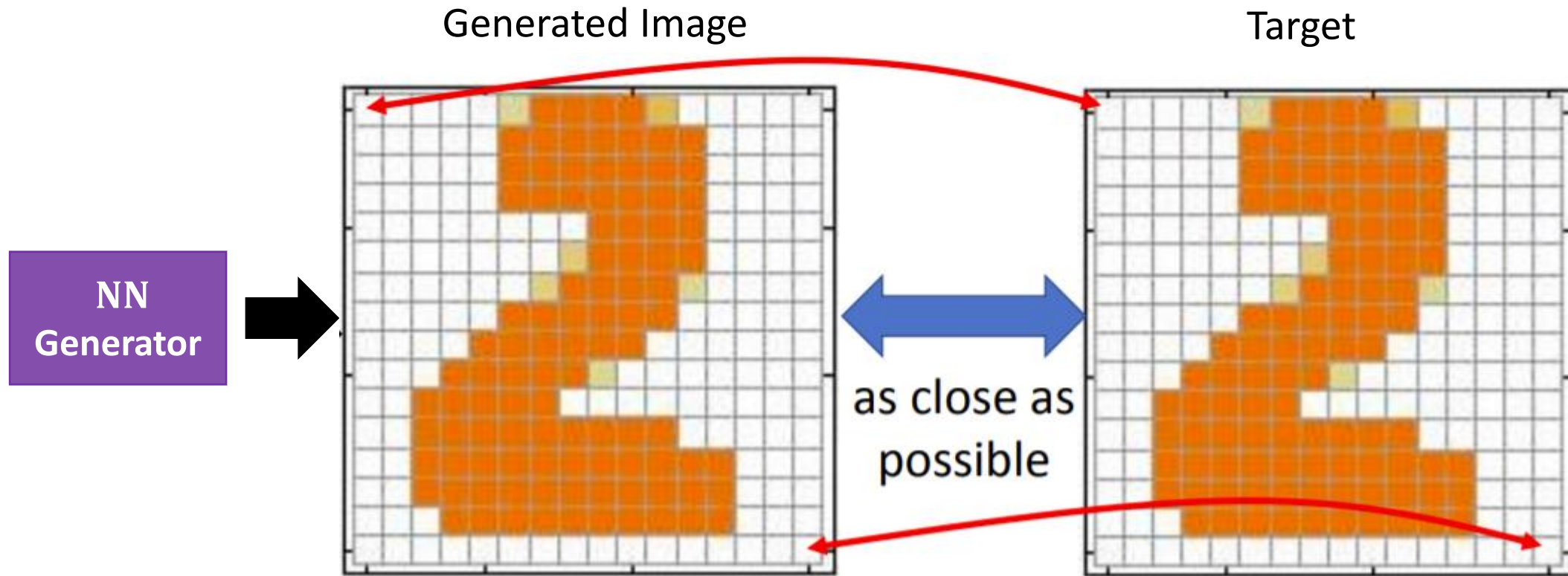


So, how we train an encoder?

Auto-encoder



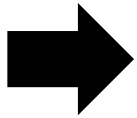
Problem of Auto-encoder in structured learning



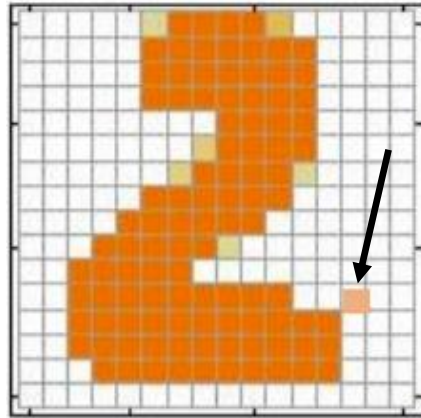
- It will be fine if the generator can truly copy the target image.
- However, this will not always be the case, and if the generator makes errors when generating an image ...

Problem of Auto-encoder in structured learning

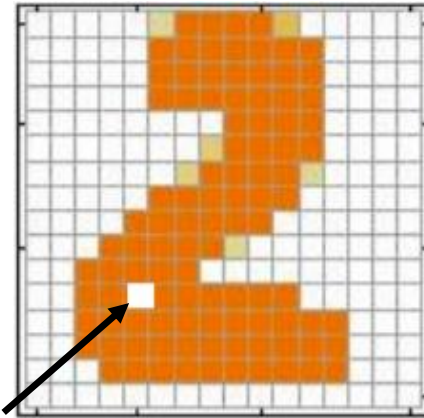
NN
Generator



Generated Images

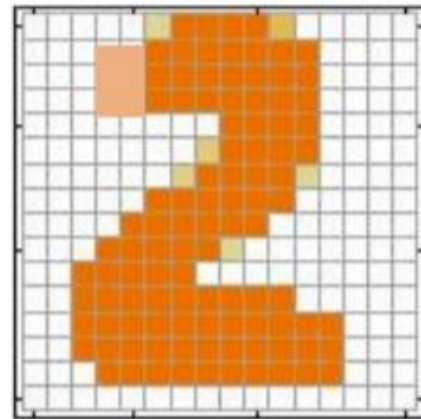


1 pixel error



1 pixel error

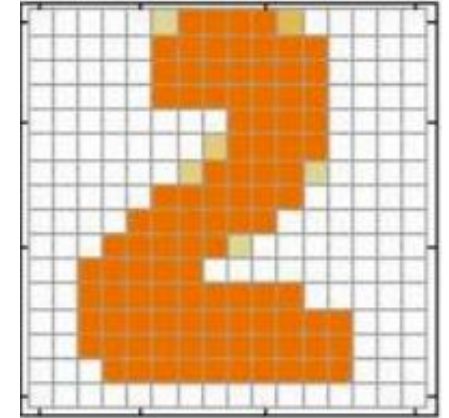
Consider
major error



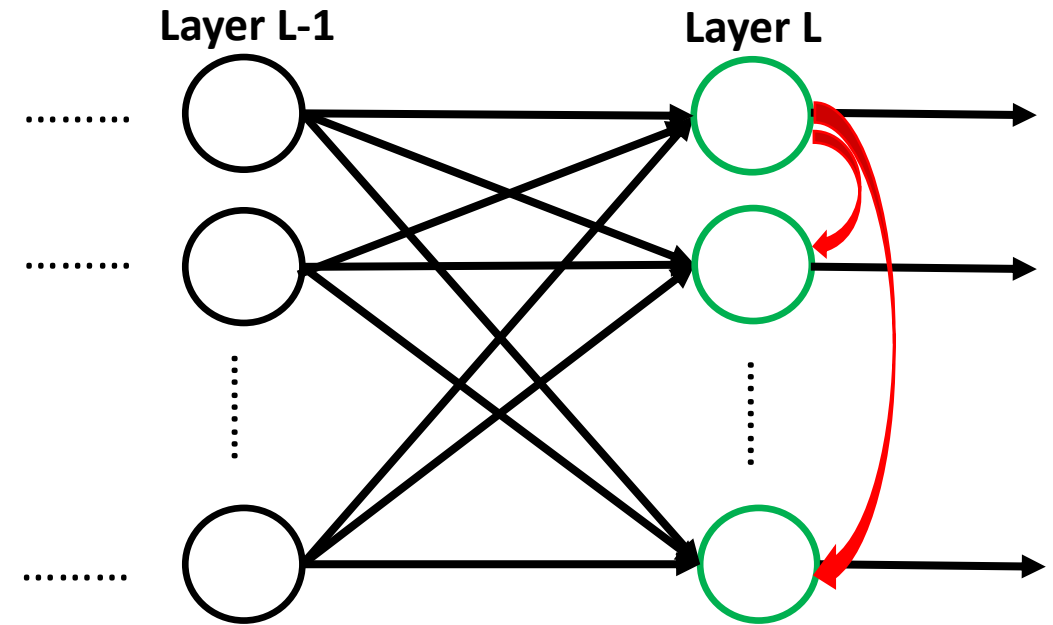
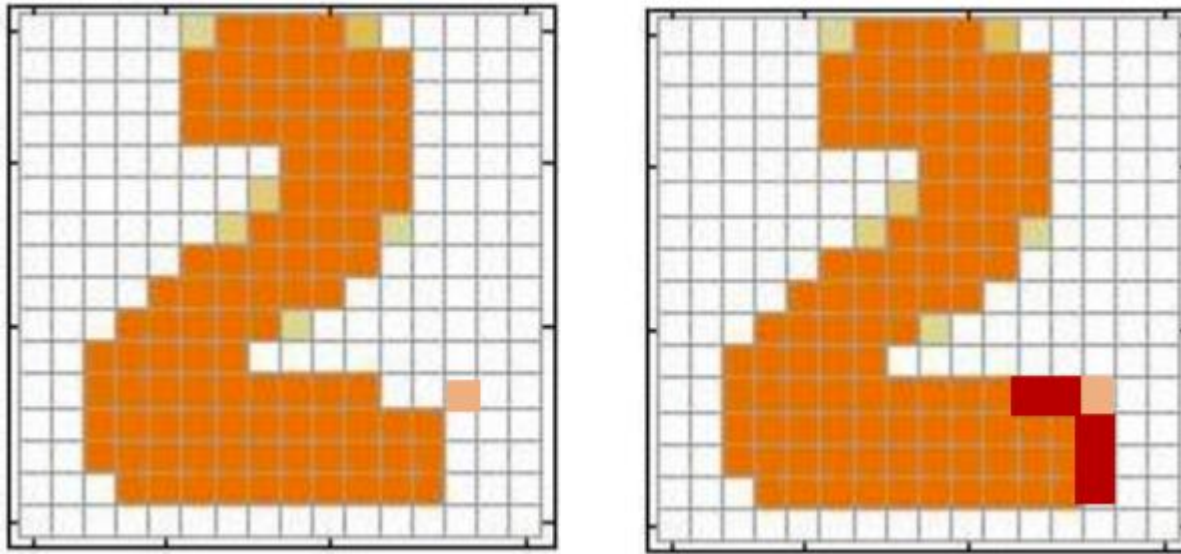
6 pixels error

Consider
minor error

Target



Problem of Auto-encoder in structured learning



- The relation between components which is critical will be missed out.
- The highly correlated components are not able to influence each other.
- Hence, we need a deep structure to catch the relation between components.

**Can Discriminator
generates image on its own?**

Discriminator

- Discriminator is a function D (network, can be deep)

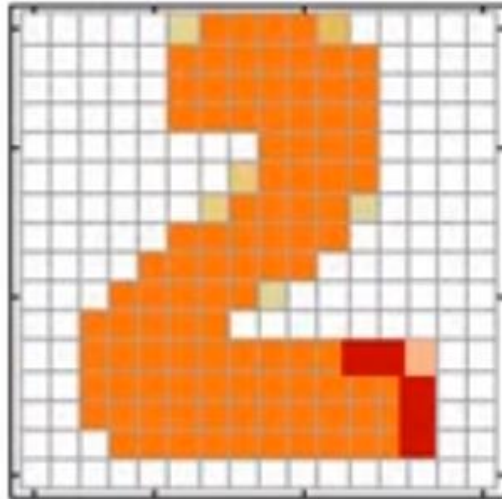
$$D: \mathcal{X} \rightarrow \mathcal{R}$$

- Input x : an object x (e.g. an image)
- Output $D(x)$: scalar which represents how “good” an object x is.



Discriminator

- It is easier to catch the relation between the components **by top-down evaluation**.

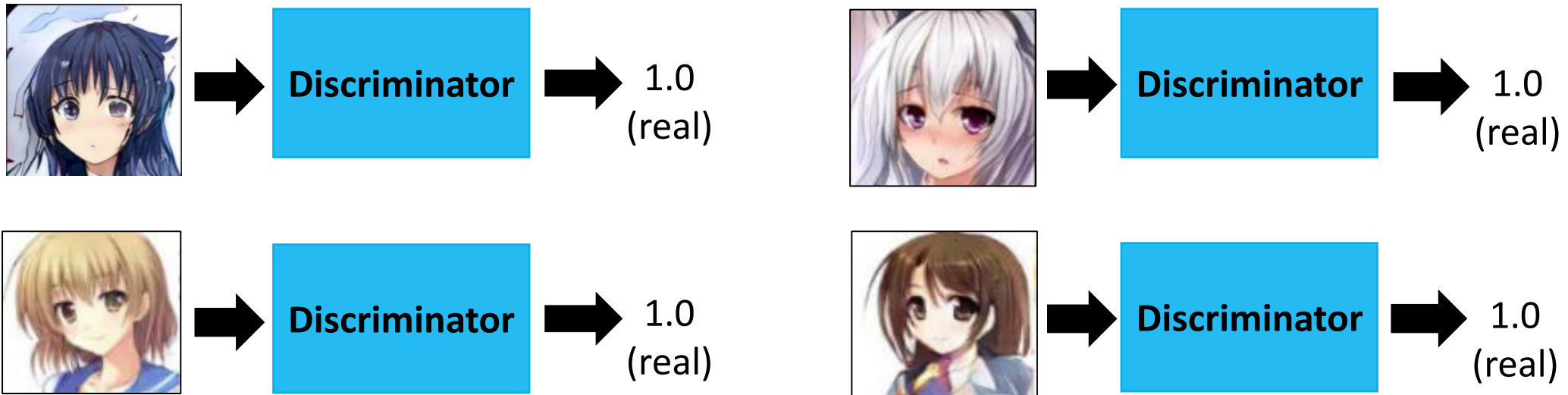


Through learnable CNN filters to check for isolated pixels.

But how to learn a discriminator?

Discriminator-Training

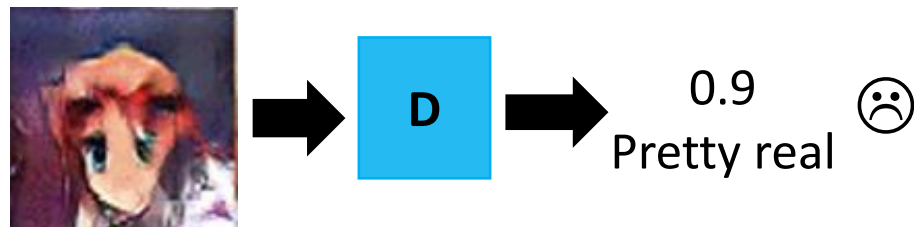
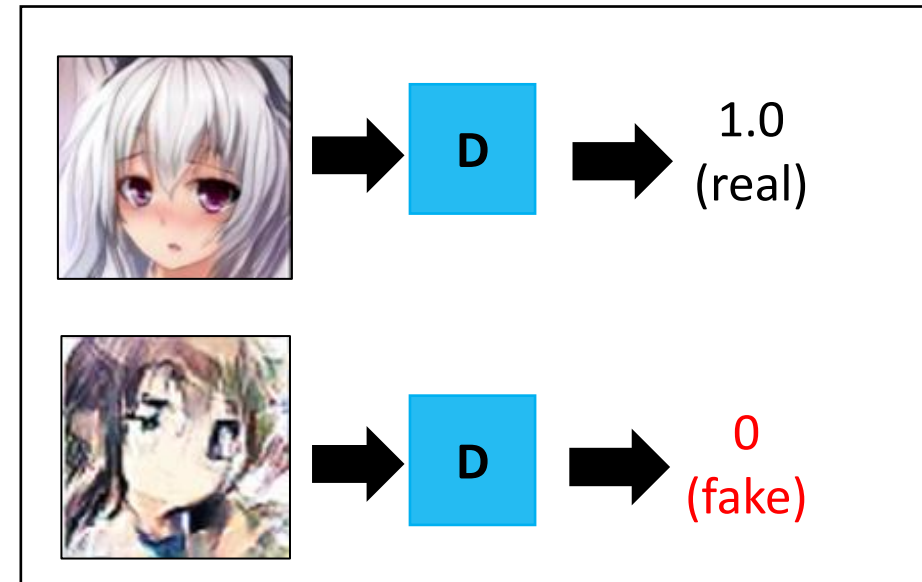
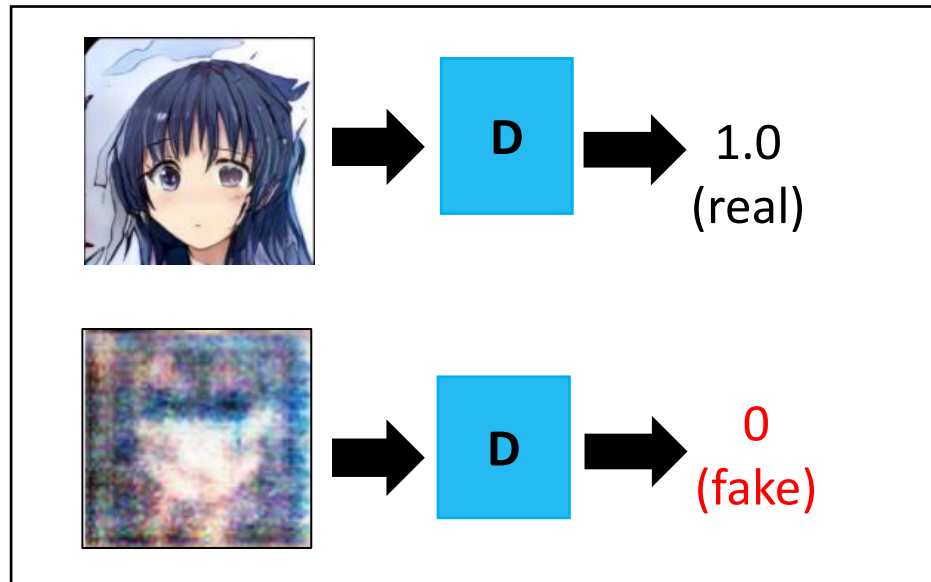
- Discriminator only manage to learn to output “1” (real) as we only have real images



Discriminator training need some negative samples

Discriminator-Training

- Negative examples are critical. But, how to produce/decide negative samples?



How to generate realistic negative samples?

Discriminator-Training

- General Algorithm
 - Given a set of **positive example**, and randomly generate a set of **negative examples**.
 - In each iteration
 - Learn a discriminator D that can discriminate positive and negative examples.
 - Generate negative example by discriminator by solving $\tilde{x} = \mathit{argmax}_{x \in X} D(x)$, which aims to maximize the score of classifying real and generated images.



However, this function turns out to be **too complicated** to address if we do not define any limitation. Since this limitation will also restrict the model's capacity, people find it easier to replace solving the $\mathit{argmax} D(x)$ function with a separate generator.

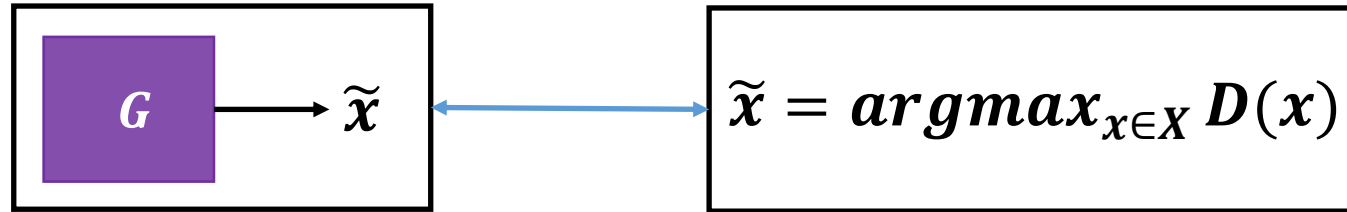
Generative model

- Pros
 - Easy to generate especially with deep model
- Cons
 - Imitate the appearance
 - Hard to learn the correlation between components

Discriminative model

- Pros
 - Considering the big picture
- Cons
 - Generation is not always feasible
 - How to do negative sampling?

Benefit of GAN



- From **Discriminator's** point of view
 - Using generator to generate negative samples
- From **Generator's** point of view
 - Still generate the object component-by-component
 - But it is learned from the discriminator with global view

Conditional Generation by GAN

- A limitation of a GAN model is that it may generate a random image from the domain. There is a relationship between points in the latent space to the generated images, but this relationship is complex and hard to map.
- Some datasets have additional information, such as a class label, and it is desirable to make use of this information.
- There are two motivations for making use of the class label information in a GAN model.
 - Improve the GAN.
 - Targeted Image Generation.

Text-to-Image

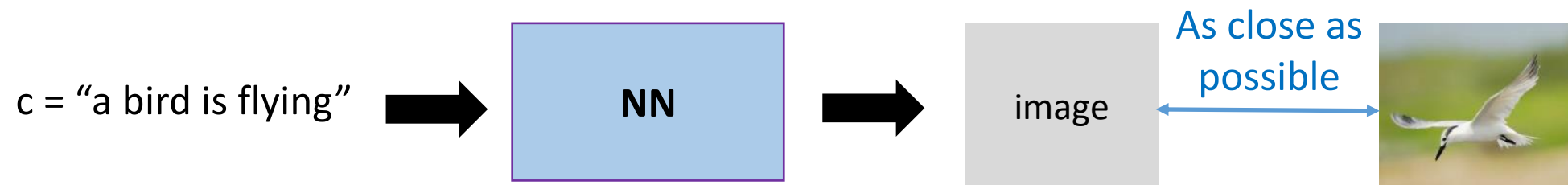
- Traditional supervised learning



A bird is flying

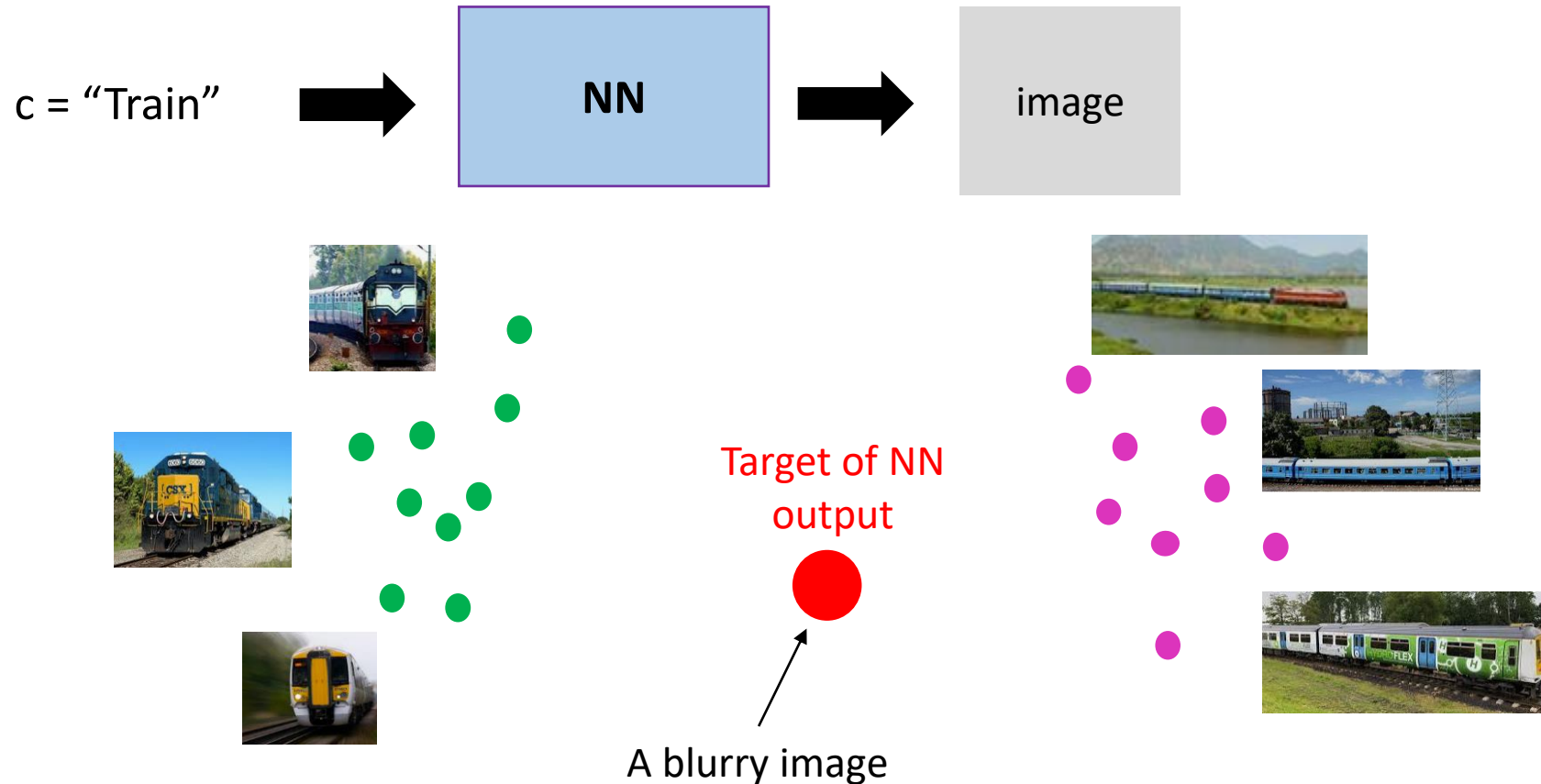


A dog is running



Text-to-Image

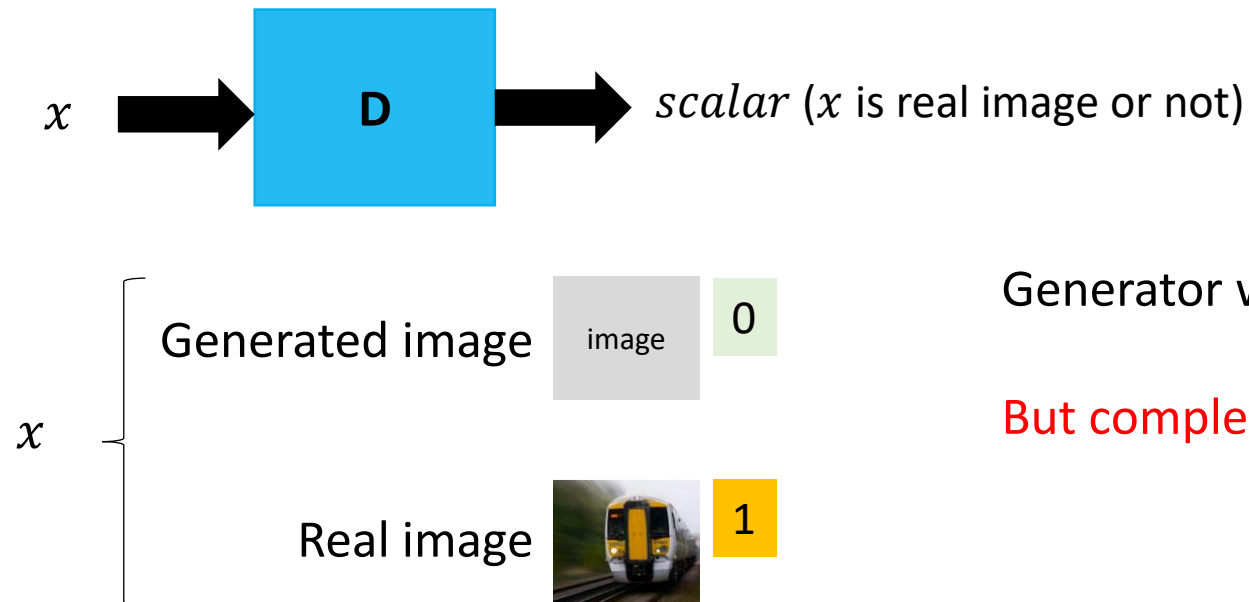
- Traditional supervised learning's problem
 - A greater possibility to produce blurred images. Blurred images may be due to the average of several images.



Conditional GAN



If discriminator model trained in such way:



Generator will learn to generate realistic images.

But completely ignore the input conditions

Conditional GAN



If discriminator model trained in such way:

Discriminator takes
in two inputs which
are the input and
output of generator.

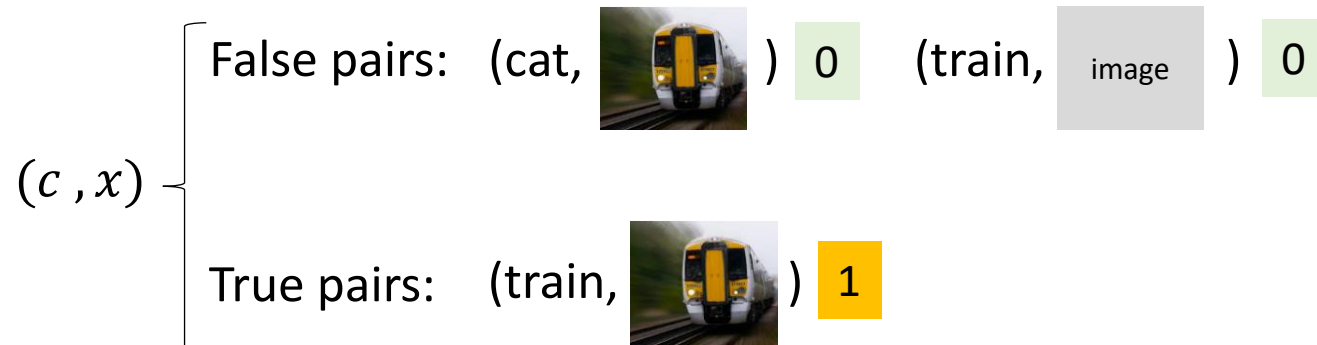
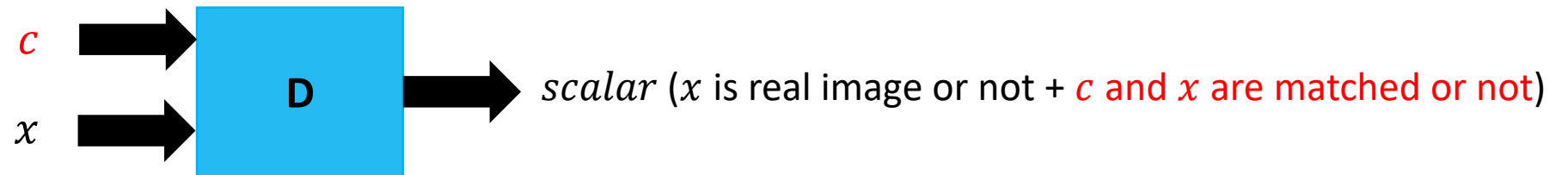
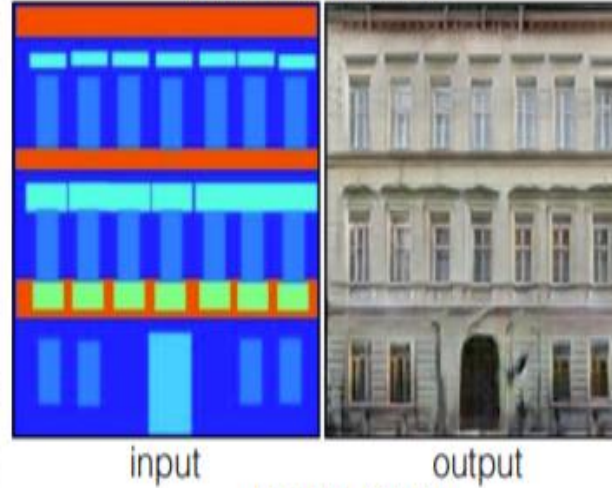


Image-to-Image

Labels to Street Scene



Labels to Facade



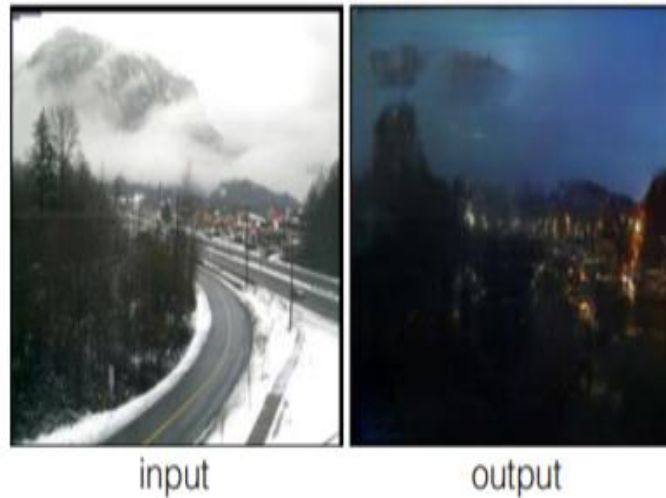
BW to Color



Aerial to Map



Day to Night

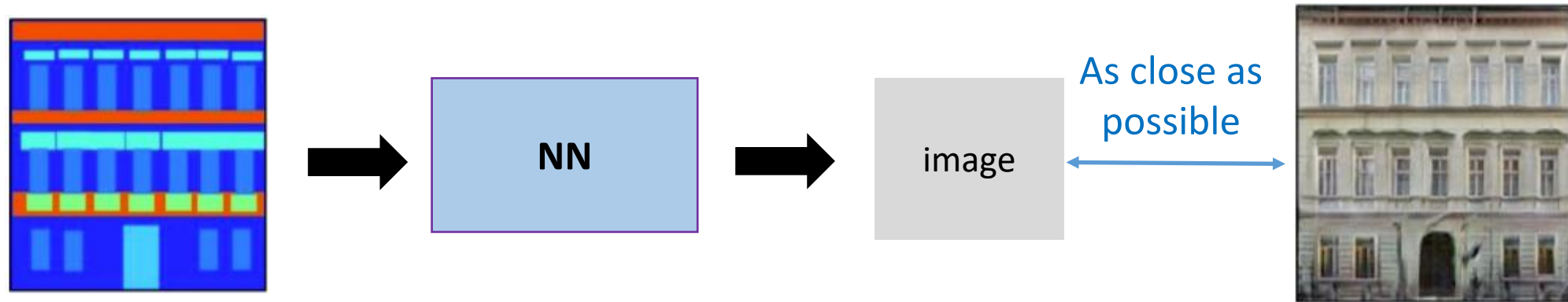


Edges to Photo

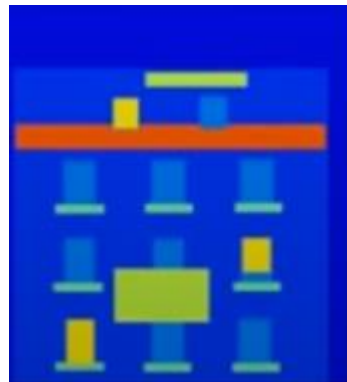


Image-to-Image

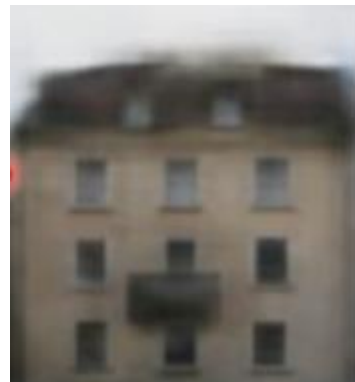
- Traditional supervised approach can be used but generated image will be blurry.



Testing:



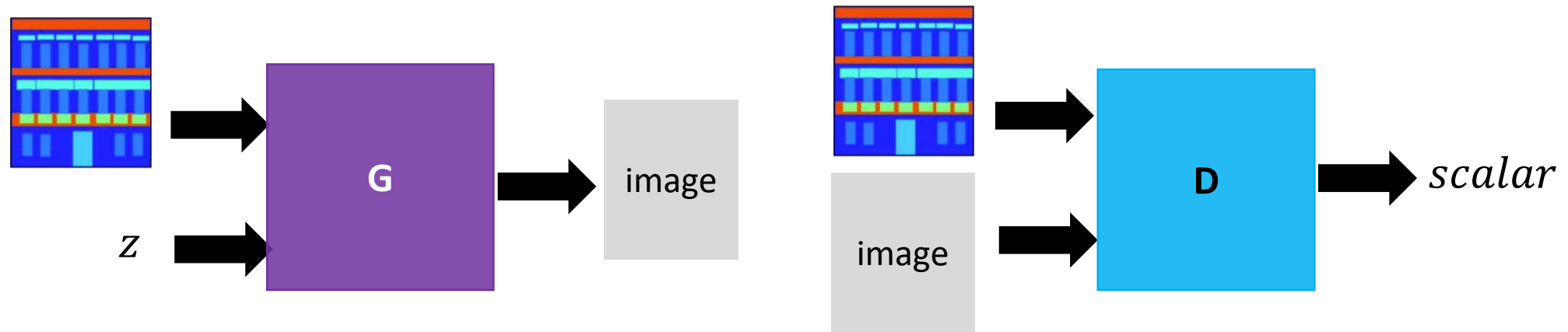
Input



Close

It is blurred because it may be due to the average of several images.

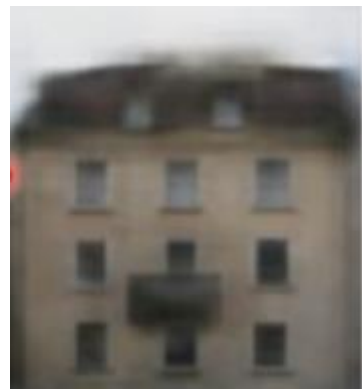
Image-to-Image using GAN



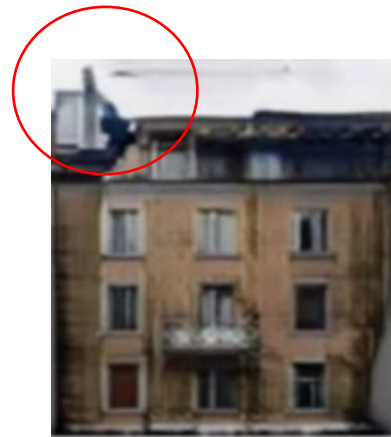
Testing:



Input

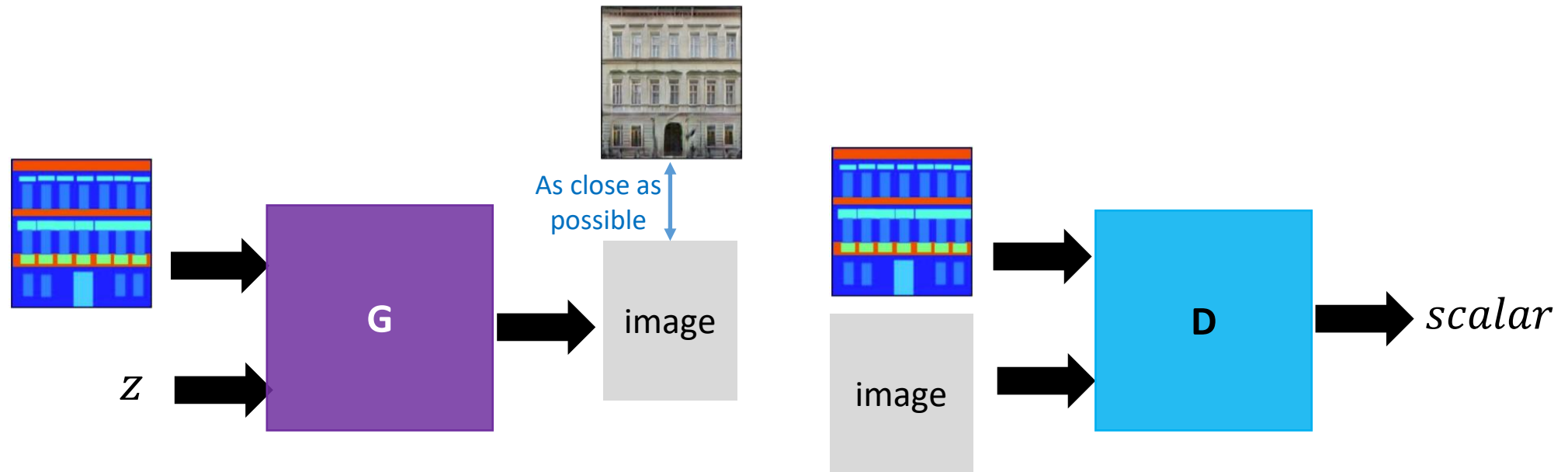


Close



GAN

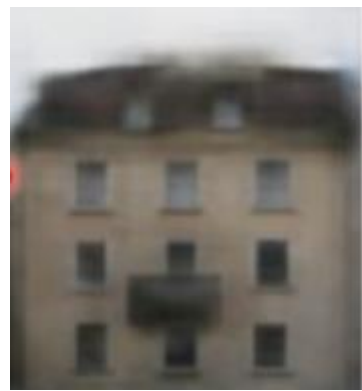
Image-to-Image using GAN



Testing:



Input



Traditional



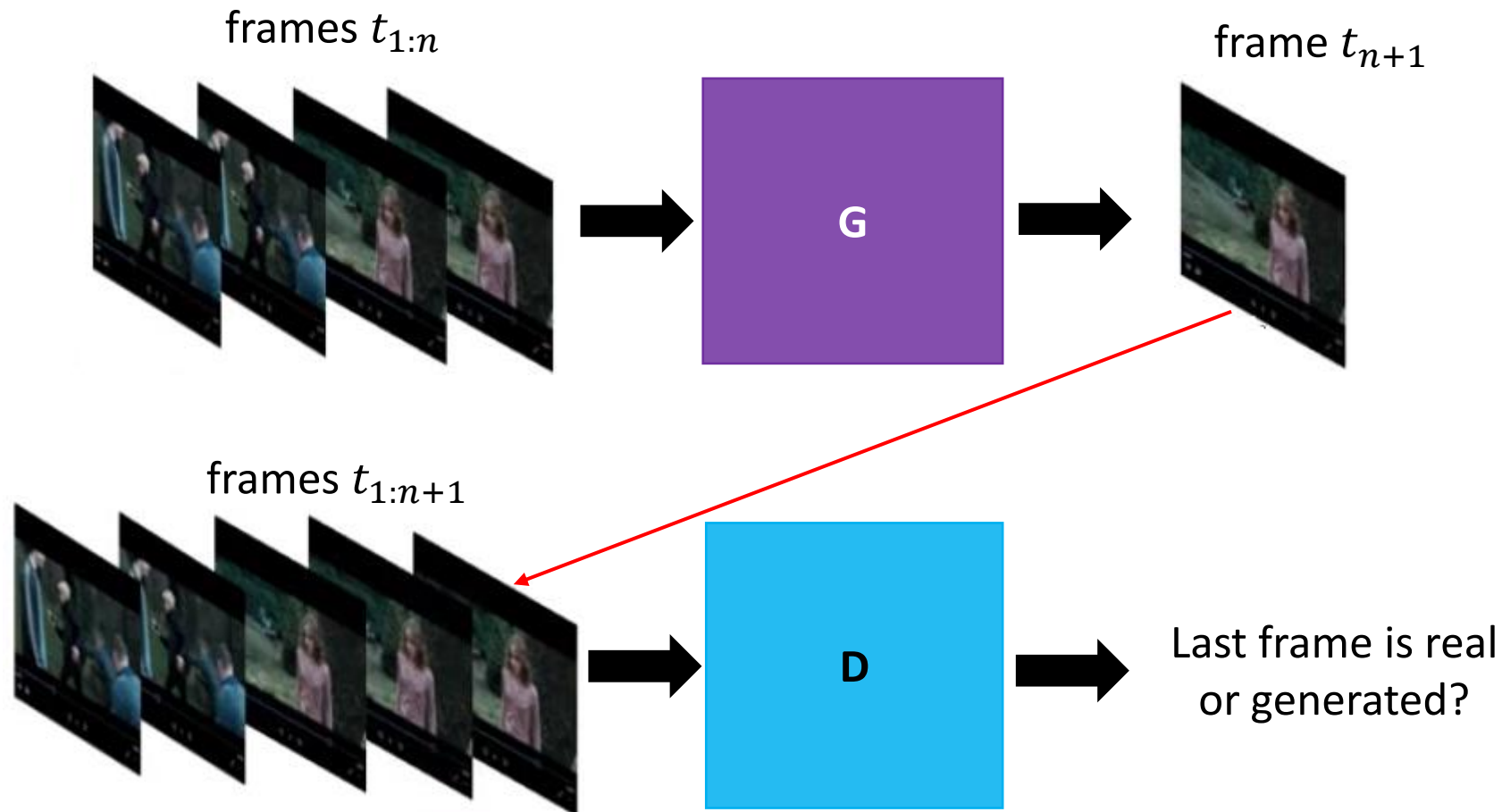
GAN



GAN + close

Other Applications using conditional GAN

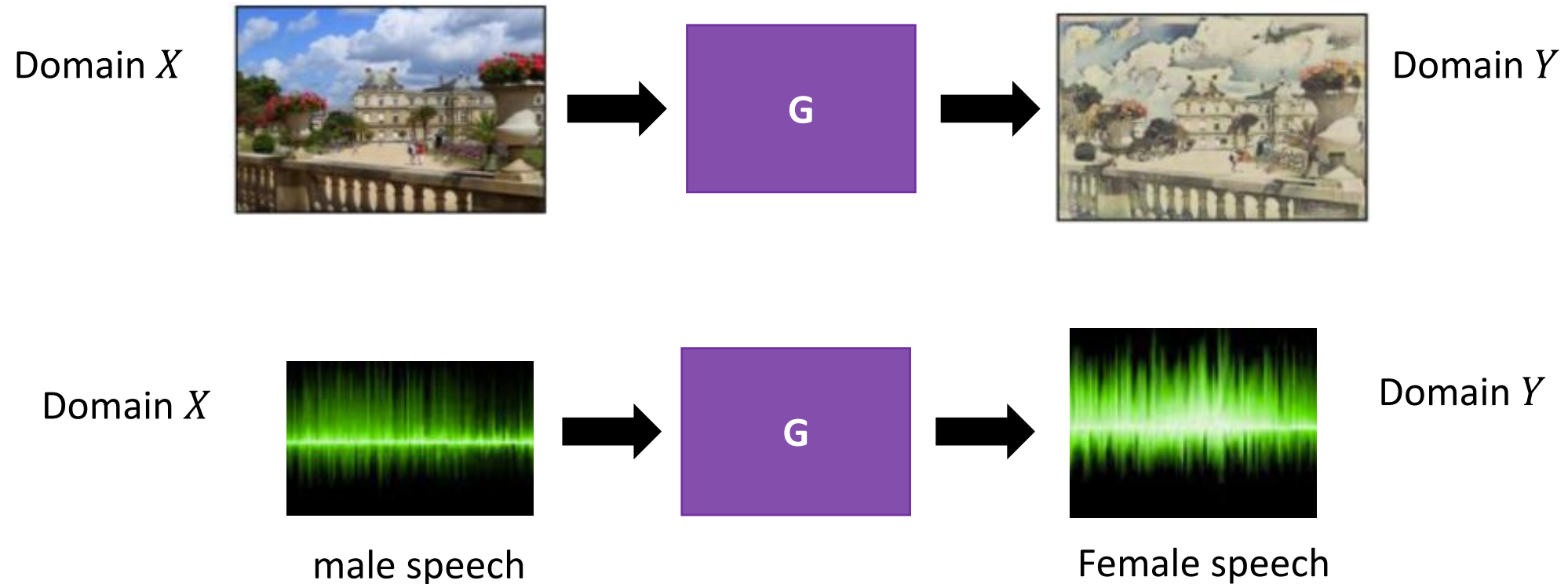
- Video Generation



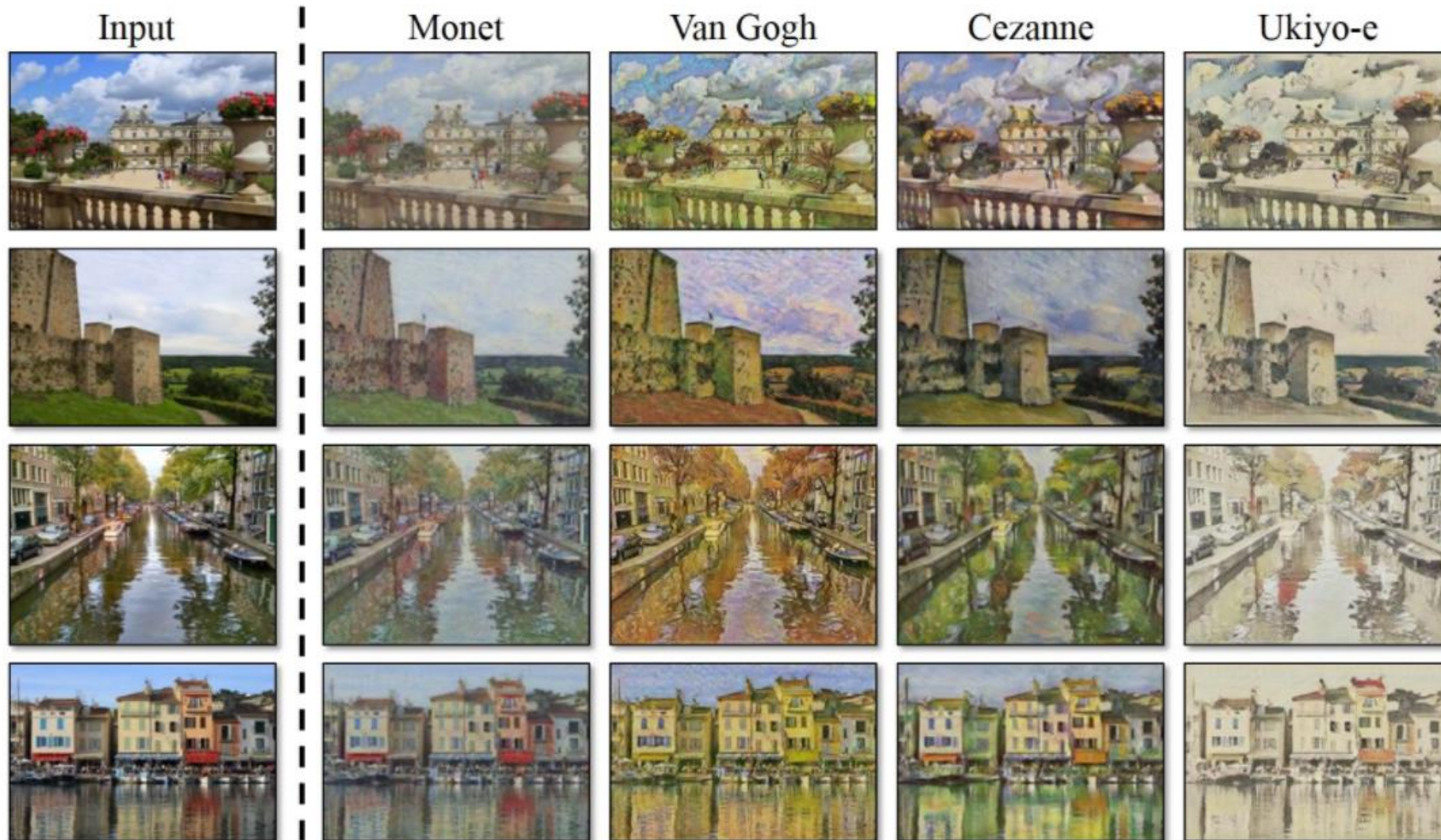
Unsupervised Conditional Generation

Unsupervised conditional generation

- Transform an object from one domain to another **without paired data** (e.g.: style transfer)

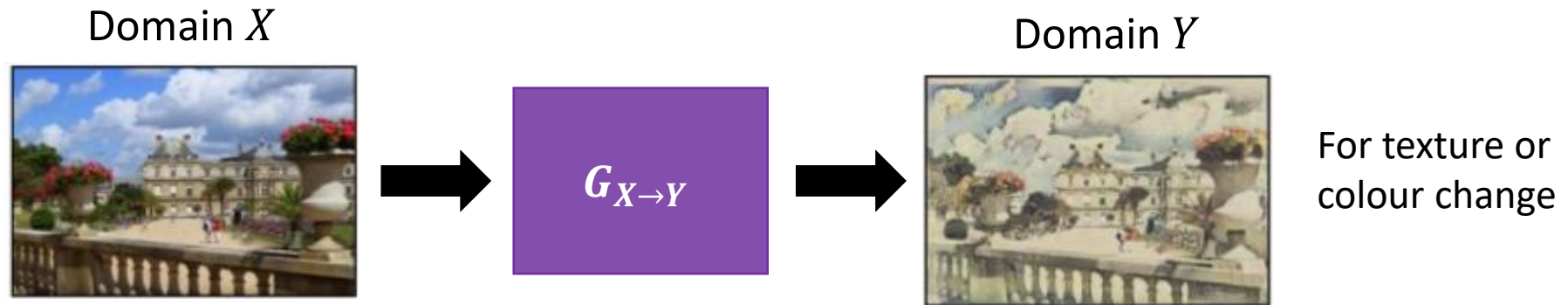


Style transfer experimental results

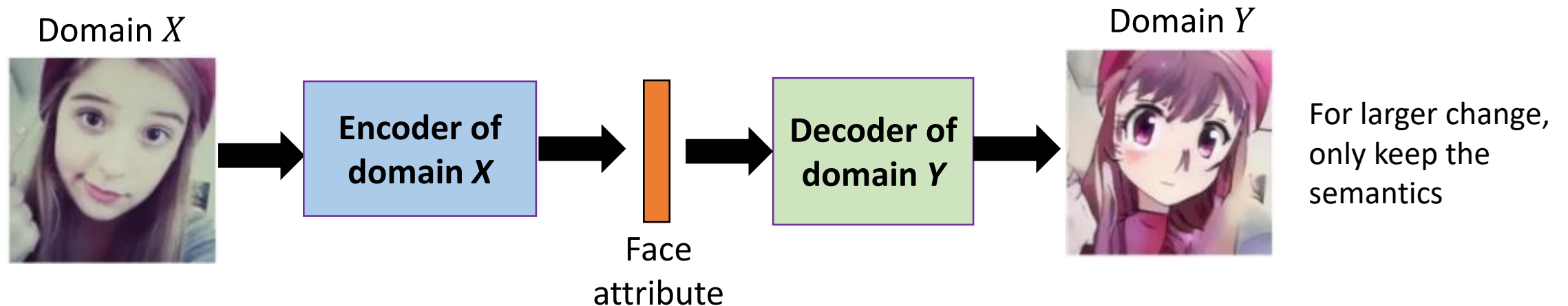


Approaches

- Approach 1: Direct transformation

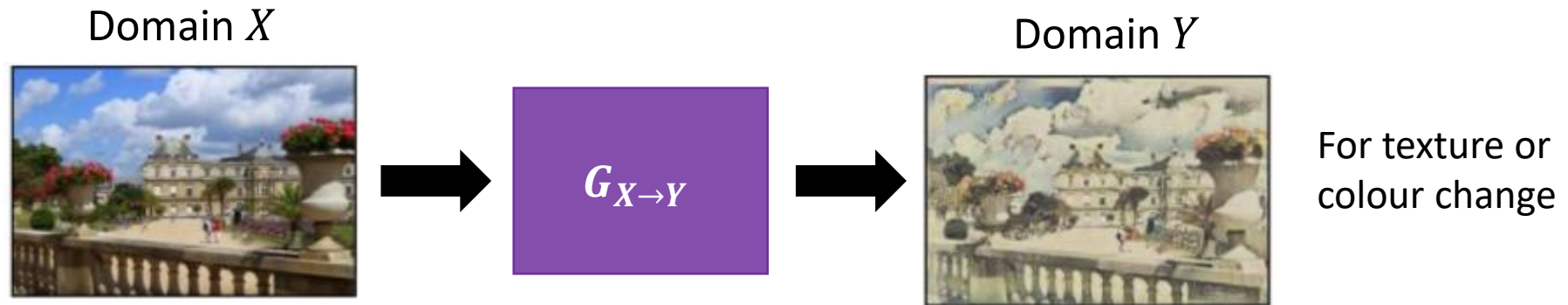


- Approach 2: Projection to Common Space

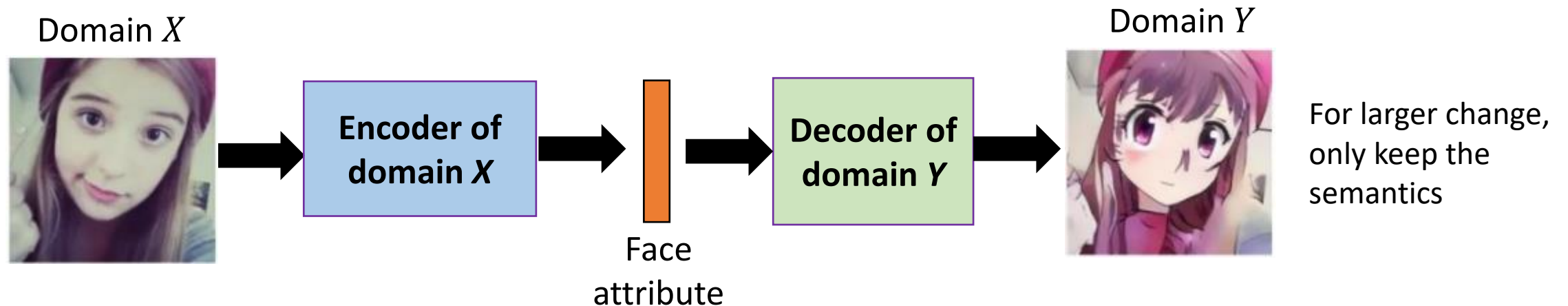


Approaches

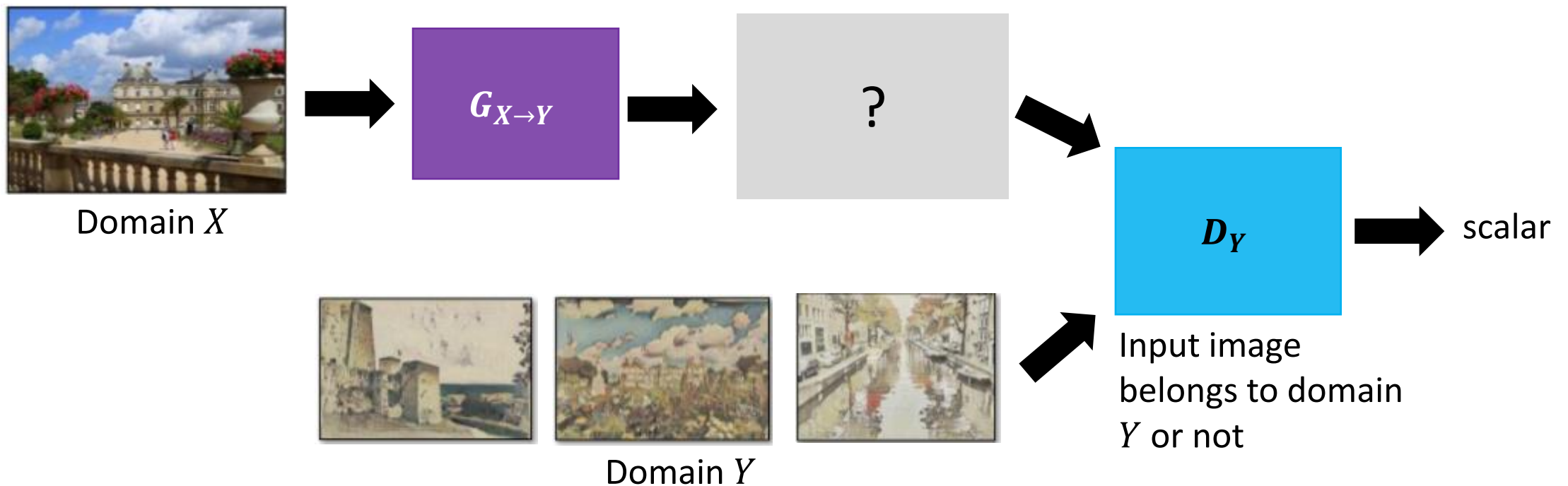
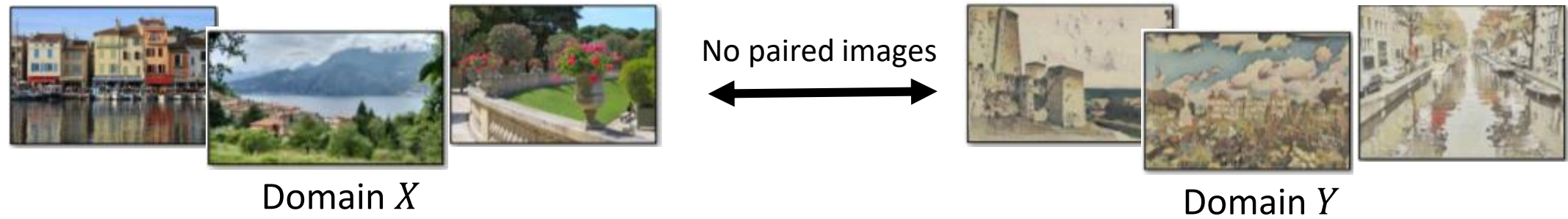
- Approach 1: Direct transformation



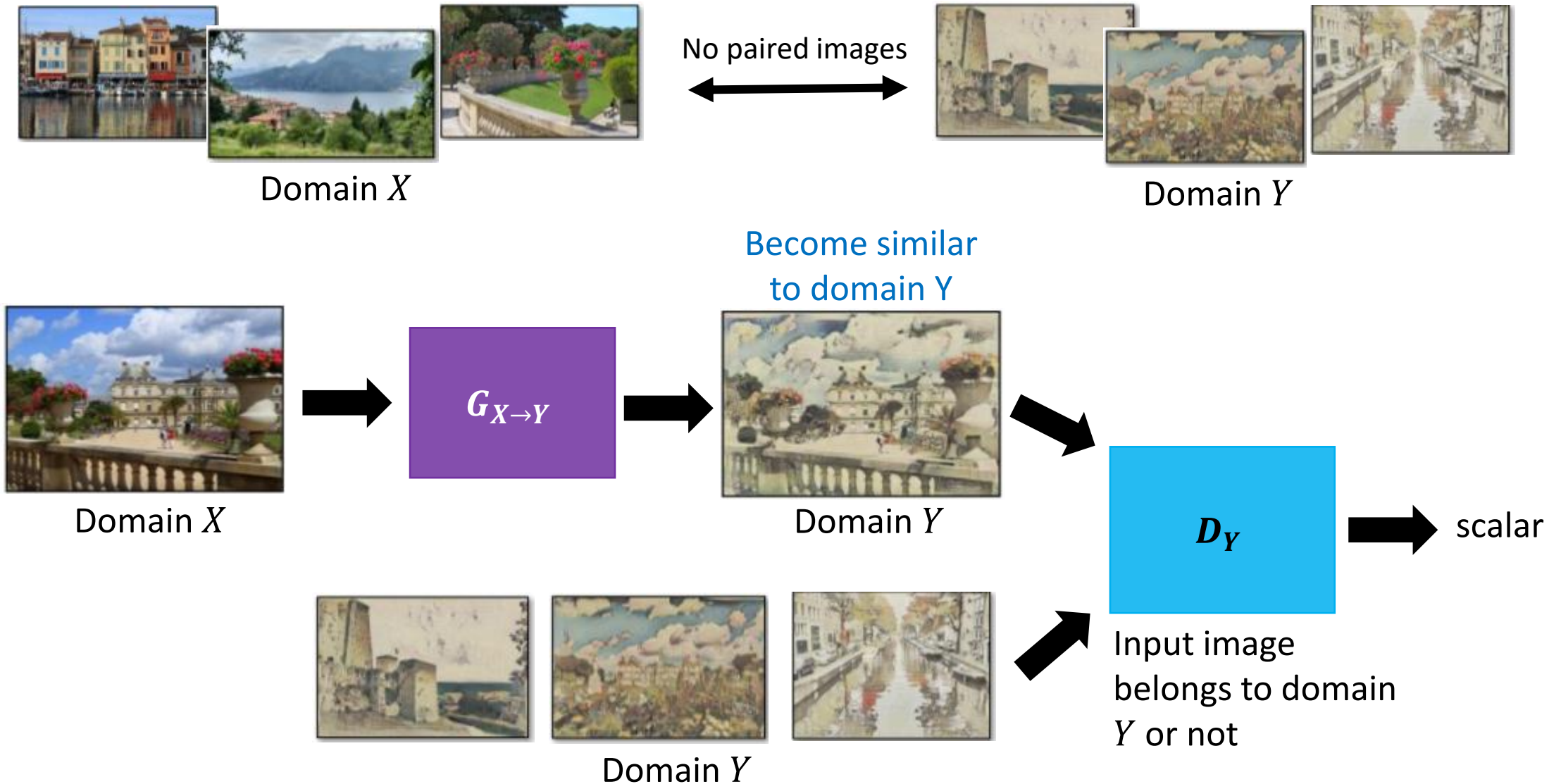
- Approach 2: Projection to Common Space



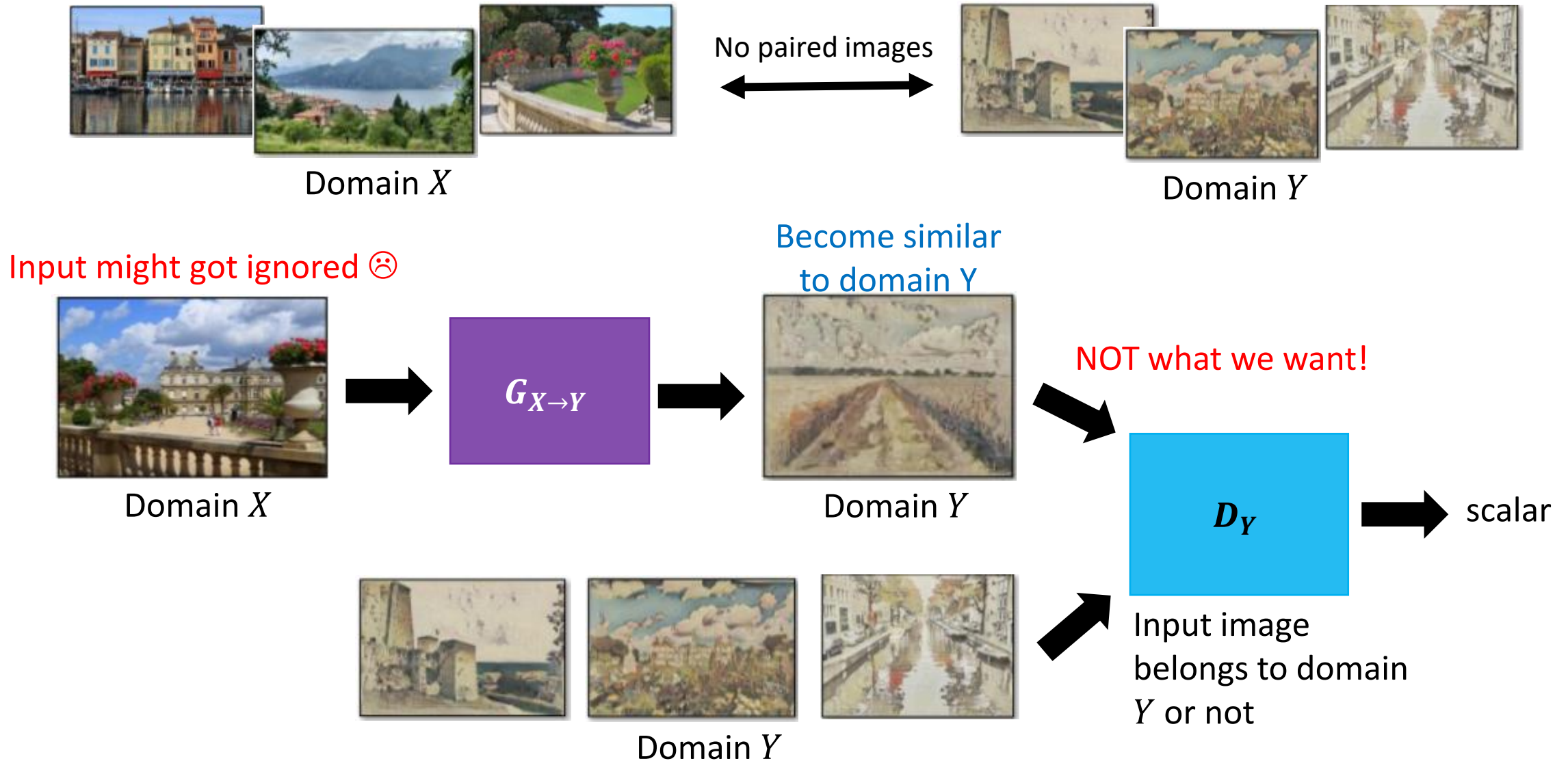
Direct transformation



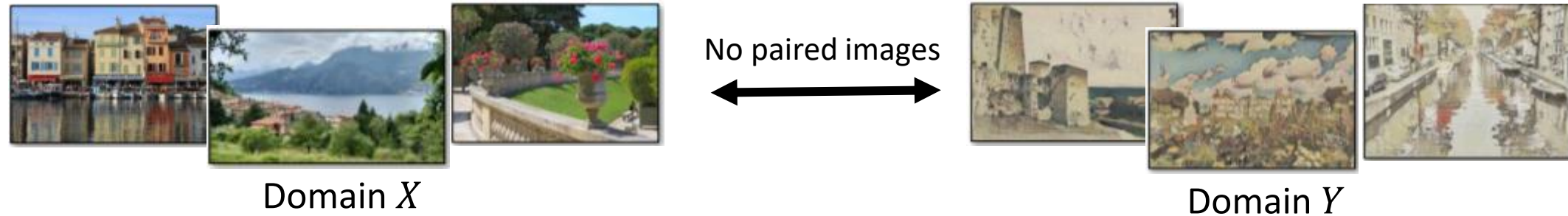
Direct transformation



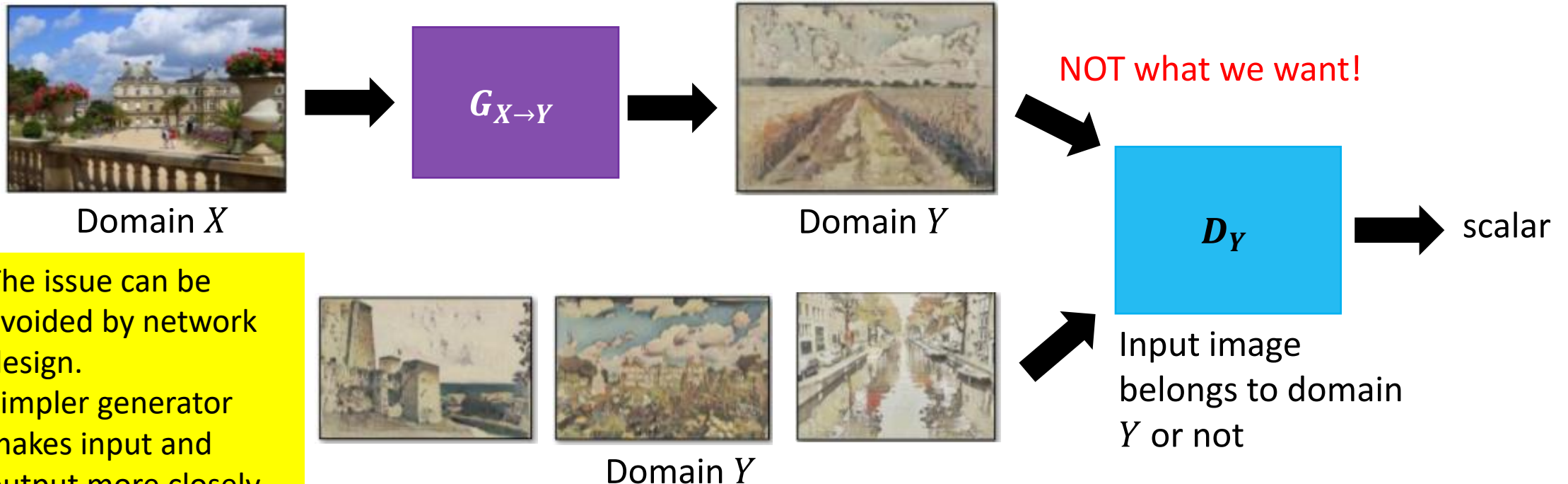
Direct transformation



Direct transformation

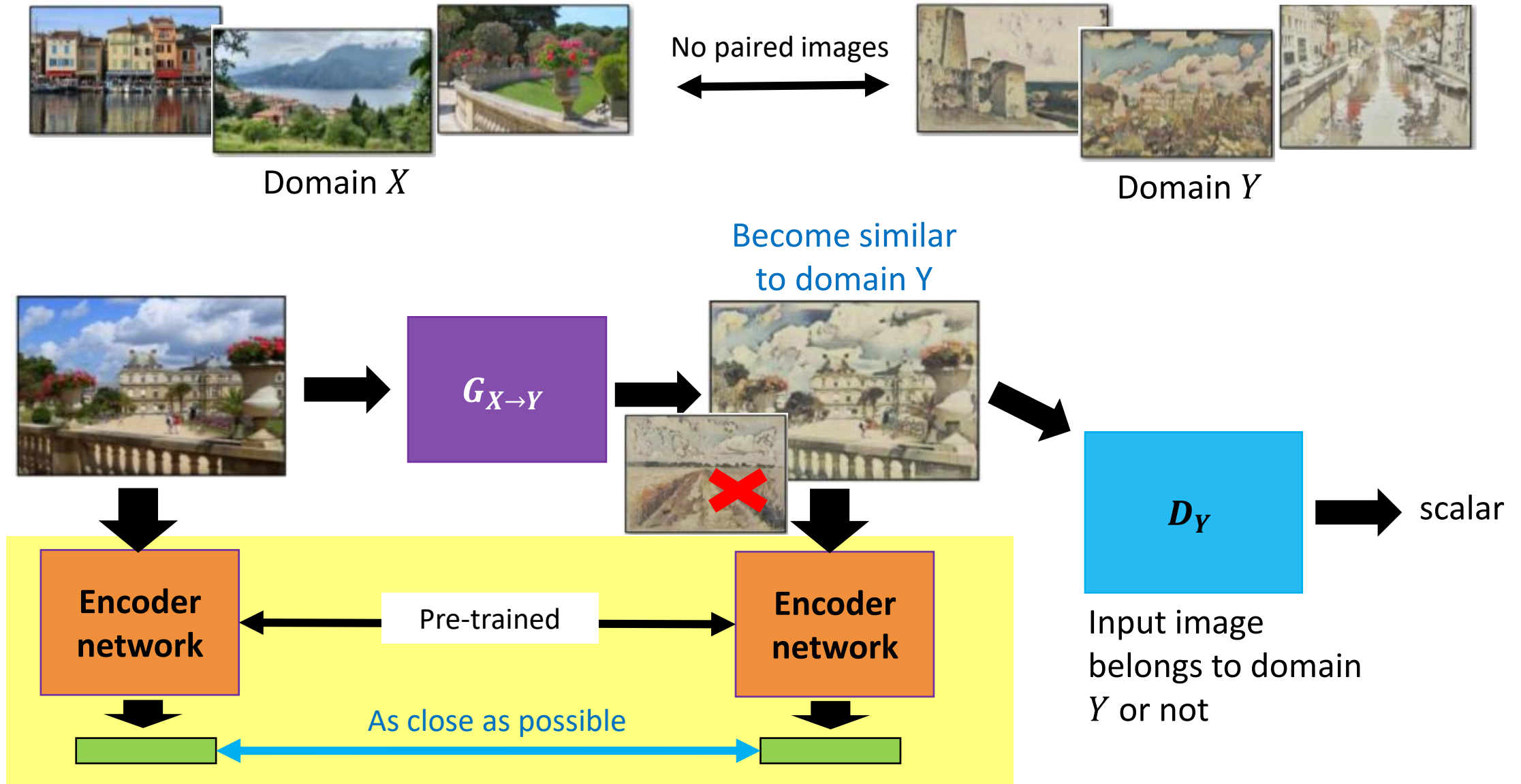


Input might got ignored ☹️

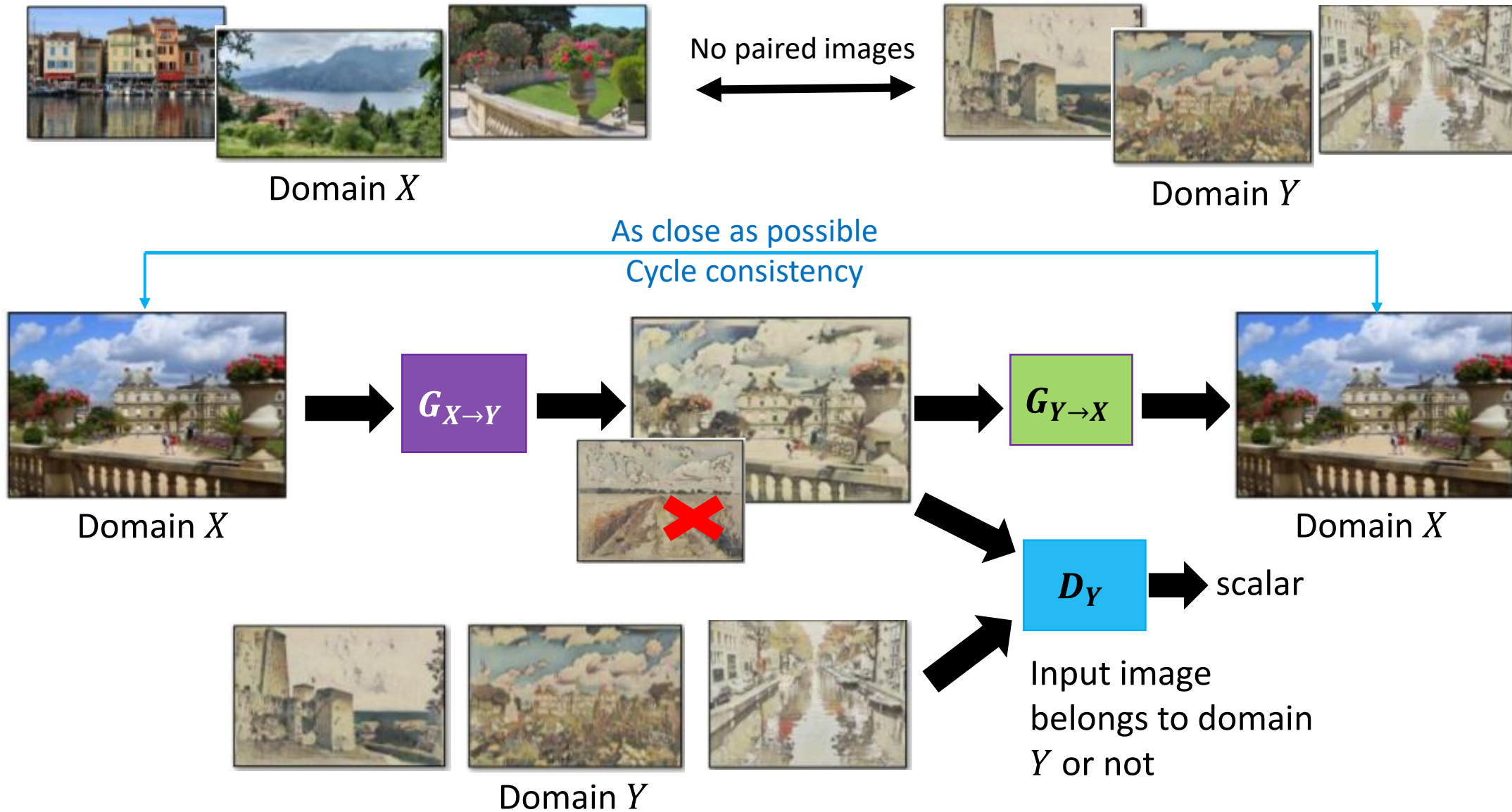


- The issue can be avoided by network design.
- Simpler generator makes input and output more closely related.

Direct transformation

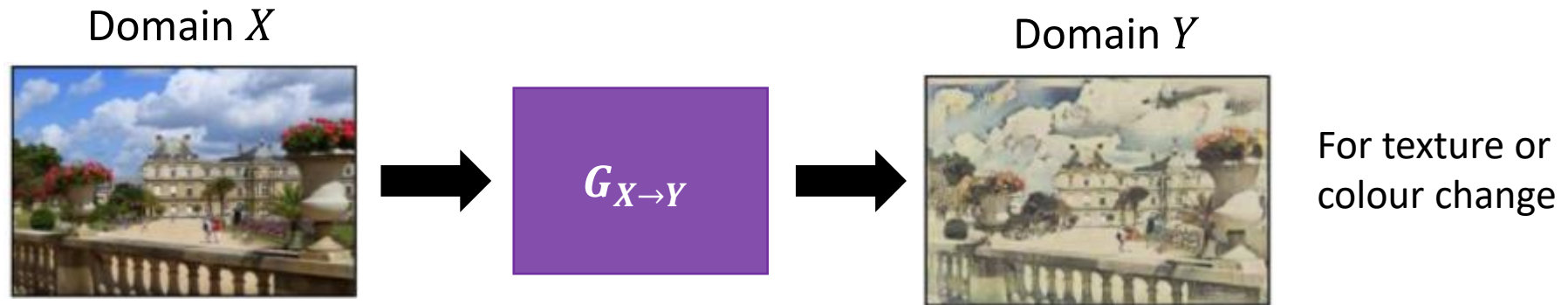


Direct transformation - Cycle GAN

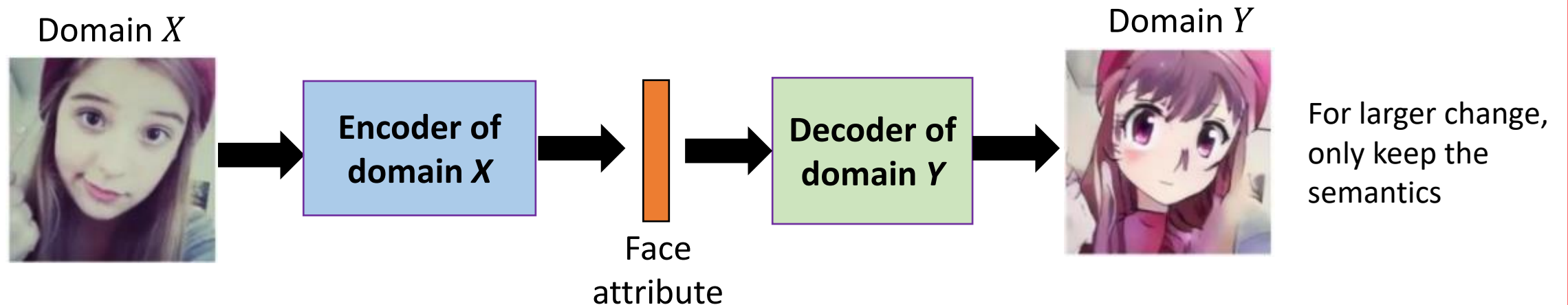


Approaches

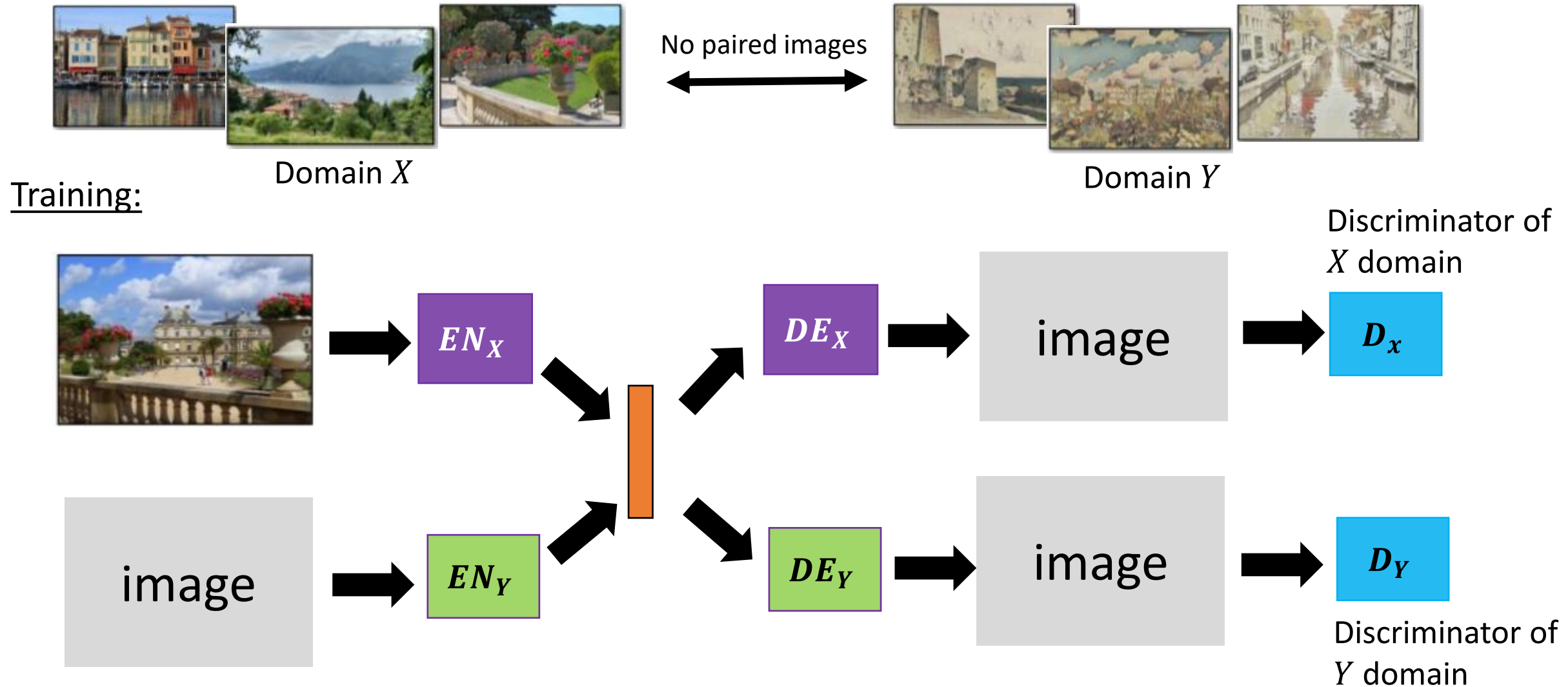
- Approach 1: Direct transformation



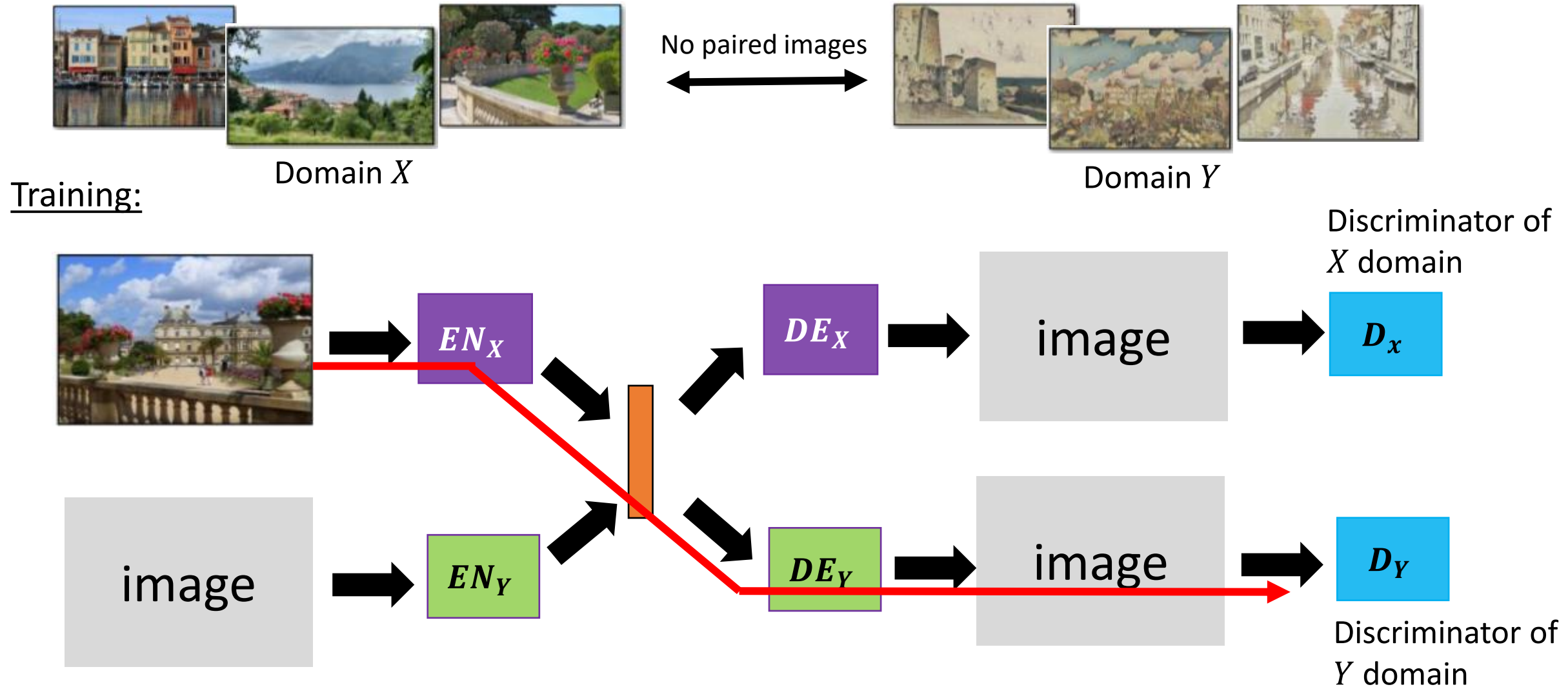
- Approach 2: Projection to Common Space



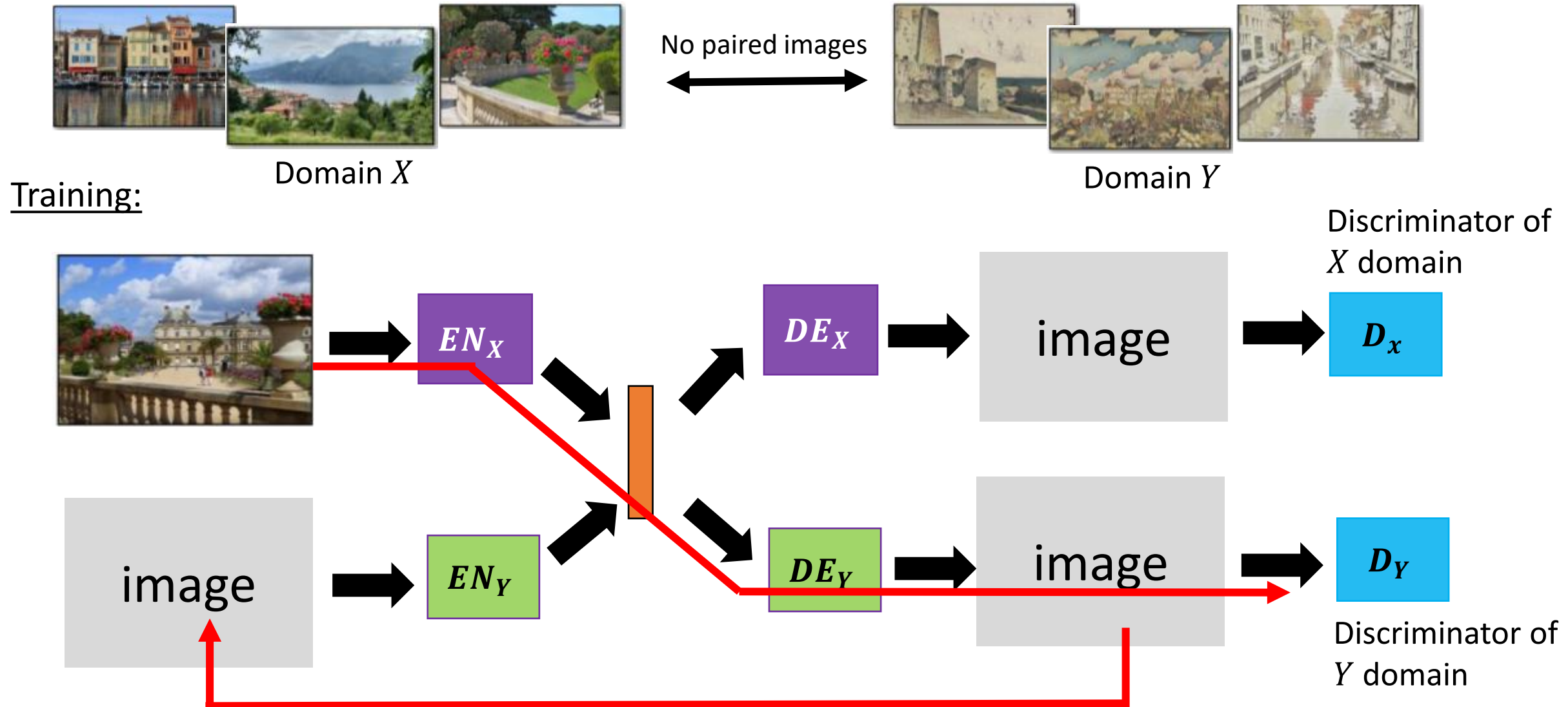
Projection to common Space – ComboGAN



Projection to common Space – ComboGAN



Projection to common Space – ComboGAN



Projection to common Space – ComboGAN



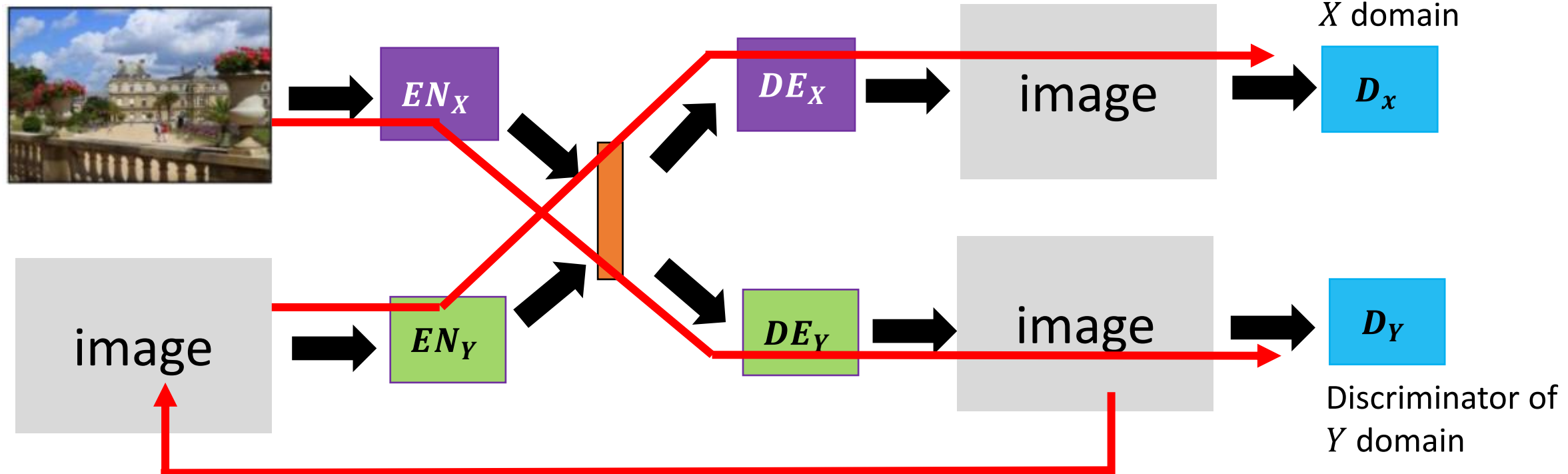
Domain X

No paired images

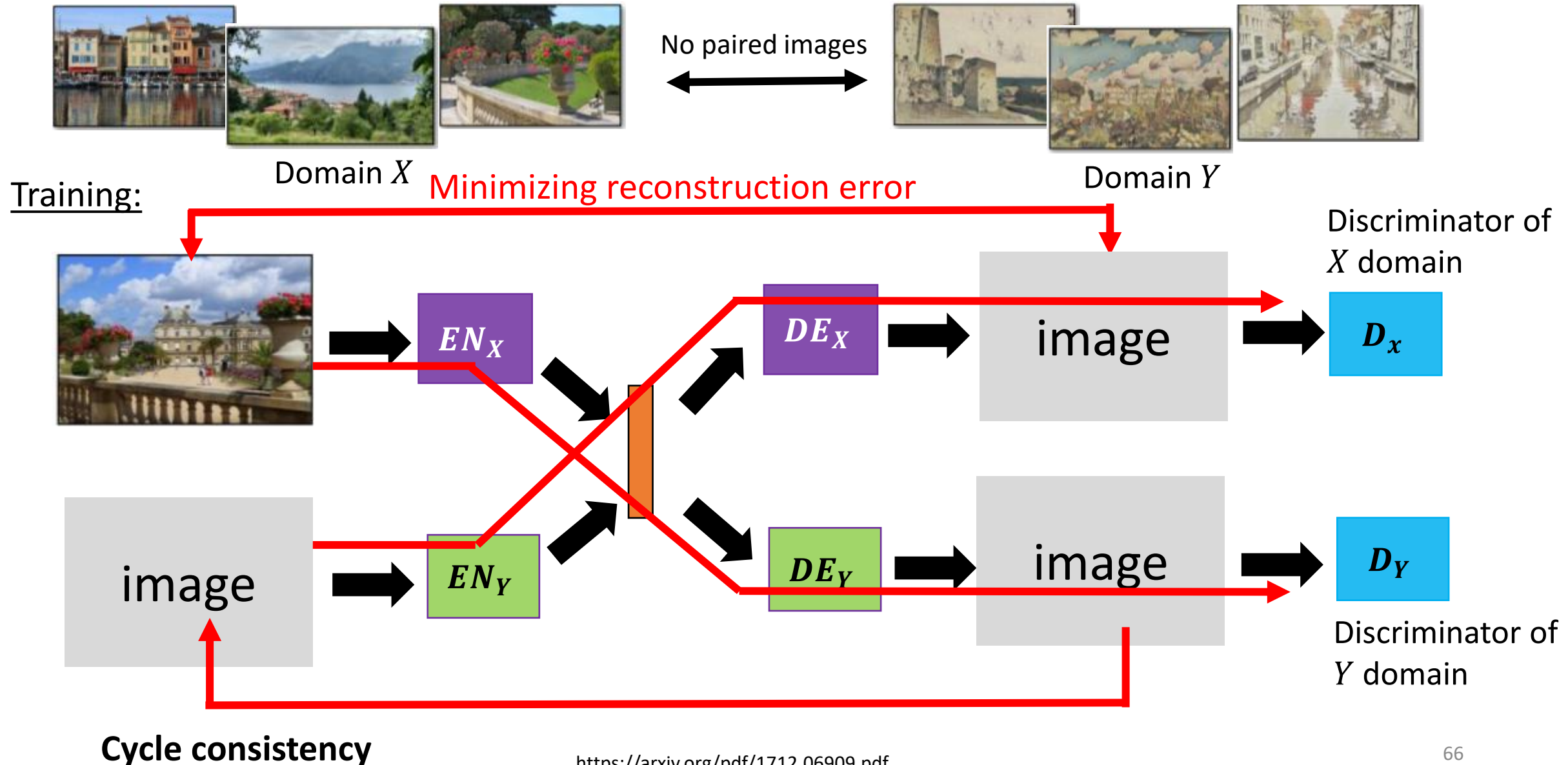


Domain Y

Training:



Projection to common Space – ComboGAN



Projection to common Space – ComboGAN



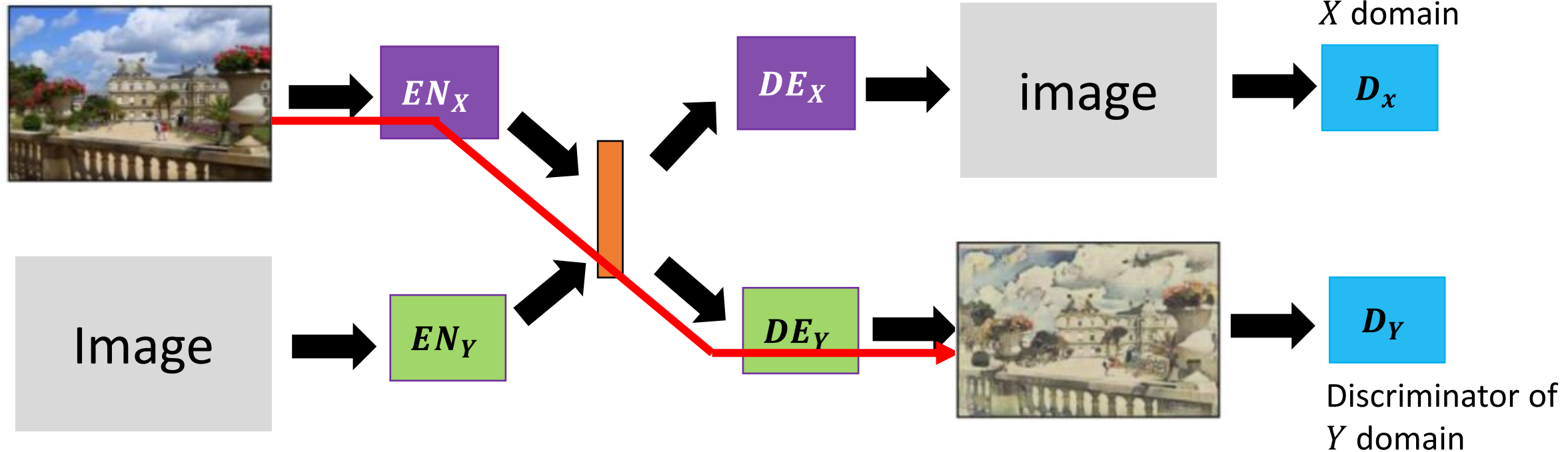
Domain X

No paired images



Domain Y

Testing:



Projection to common Space – ComboGAN



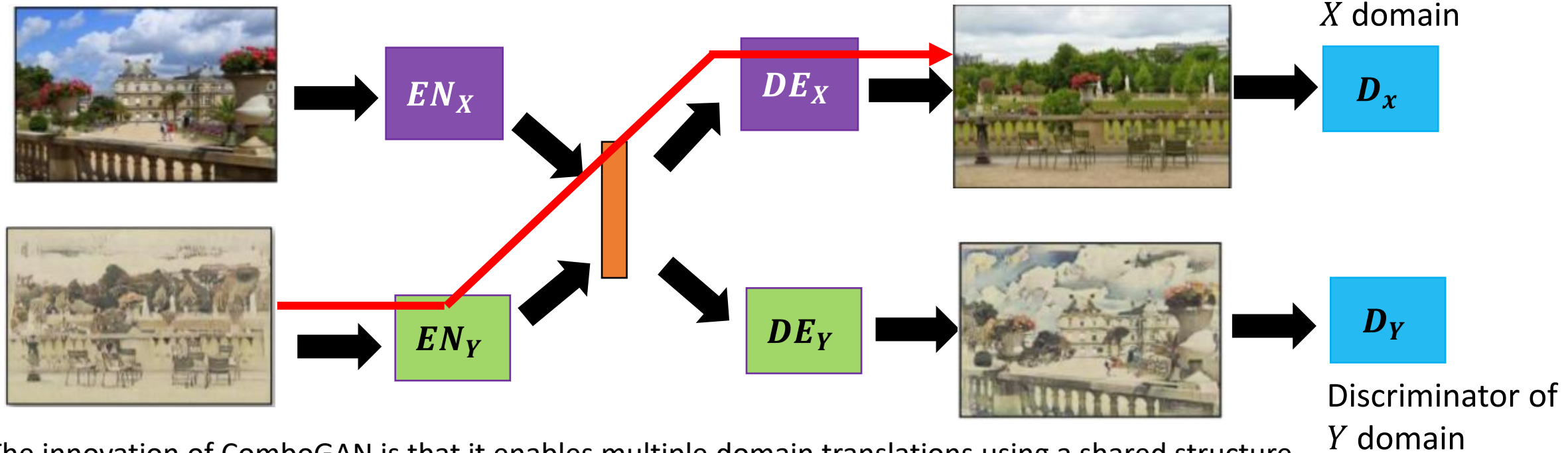
Domain X

No paired images



Domain Y

Testing:



References

- The-GAN-Zoo
 - <https://github.com/hindupuravinash/the-gan-zoo>
- Keras-GAN
 - <https://github.com/eriklindernoren/Keras-GAN>